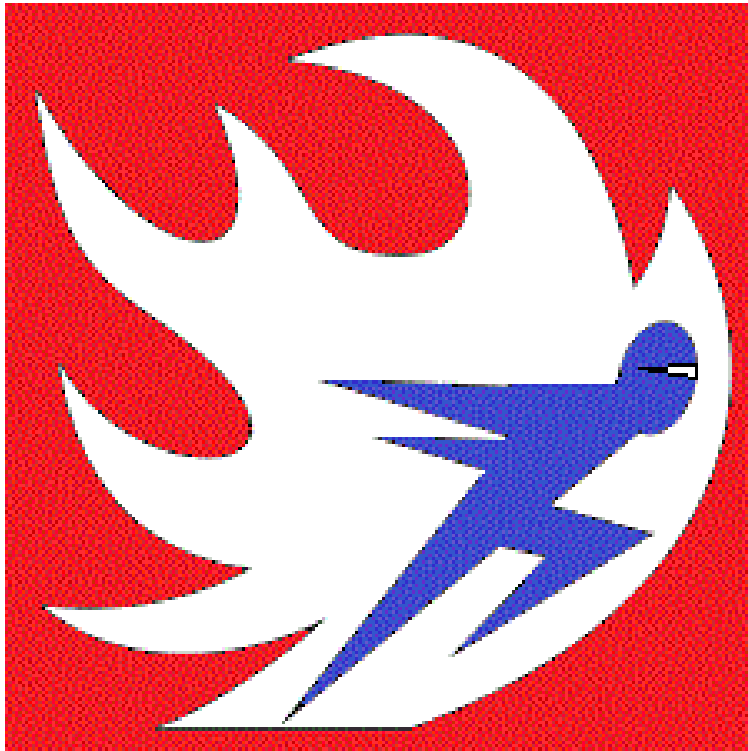


DASH TEST®



Created by: Edgar Chavez

First Draft: 2016/02/11

Latest Revision: April 2019

DASH TEST®

Introduction.

DASH TEST® is a Test Acceleration and Automation Engine that can be used to simplify tasks during System Development Functional and Regression Testing phases.

DASH TEST® is completely developed using Java JRE 1.8 programming system. Takes advantage of the Selenium Web Driver JAVA libraries, to interact with Web Pages, using Web Browser.

DASH TEST® execution is driven using Test Suite configuration files written in XML format, where parameters are passed to the main class method. Among those parameters, the user must provide the location of the Test Script and the Object Repository files.

Among the advantages of using DASH TEST® to automate your tests, is that scripts are written in an MS Excel file, step by step, in plain English that mimics your Manual Test Case. All this is accomplished by using a very simple syntax. There is no coding involved, so you don't need to know any programming language or understand complicated rules. All you need to know is having a clear knowledge of your Application Under Test (AUT), its components, the Business Processes flows, and the steps needed to achieve your validation goals.

DASH TEST® can be executed from a batch file, meaning that it can be executed through continuous integration tools such as Jenkins, just by giving it the location to your Test Suite configuration and script files.

DASH TEST® includes a user-friendly User Interface. This UI makes easy to create or modify your Test Execution configuration files and execute your Test Suites.

There are some things you must remember:

Not all tests can be automated and not all tests should be automated. Just for checking if that button is present in a screen of the AUT... do you think that needs to be automated? Better evaluate your options and decide if what tests you really must to automate.

Automation tools cannot make miracles. A library of automated scripts always starts with the first script. What did you expect? Press a button and have a full test suite ready?

Before trying to automate that test, be sure to have your manual test script with all details needed to accomplish your test goals and test data ready, DASH TEST® is an awesome tool, but it cannot guess

where to find that “Total Dollar Amount to Pay”, unless you tell how to navigate through all those 50 screens and look at the bottom of the “Payment Confirmation” page, just like you did on your first day at work.

DASH TEST® can execute tests on Google Chrome, Mozilla Firefox and Microsoft Internet Explorer Web Browsers. To run tests in any of the mentioned Web Browsers, you must have them installed in your computer. You can obtain them from their respective Web Sites.

A question that I have been getting from DASH TEST® users is “Can DASH TEST® be used for mobile testing?”, the short answer would be “Not Really”, but don’t feel disappointed, there is a longer, elaborated answer to that question. “You cannot use DASH TEST® for mobile APP testing, but there are frameworks specially created to emulate mobile websites. That’s where you can use DASH TEST®”.

To successfully run DASH_TEST you must at least have JAVA 7 installed in your computer. Refer to the Java documentation or their website to successfully install Java.

It is recommended to have FULL read and write access to your computer.

DASH TEST® is a Copy Right property of:

Edgar Chavez 2015 (PRE000008332).

Designed and Developed by Edgar Chavez.

This is Version 4, completed in April 2019.

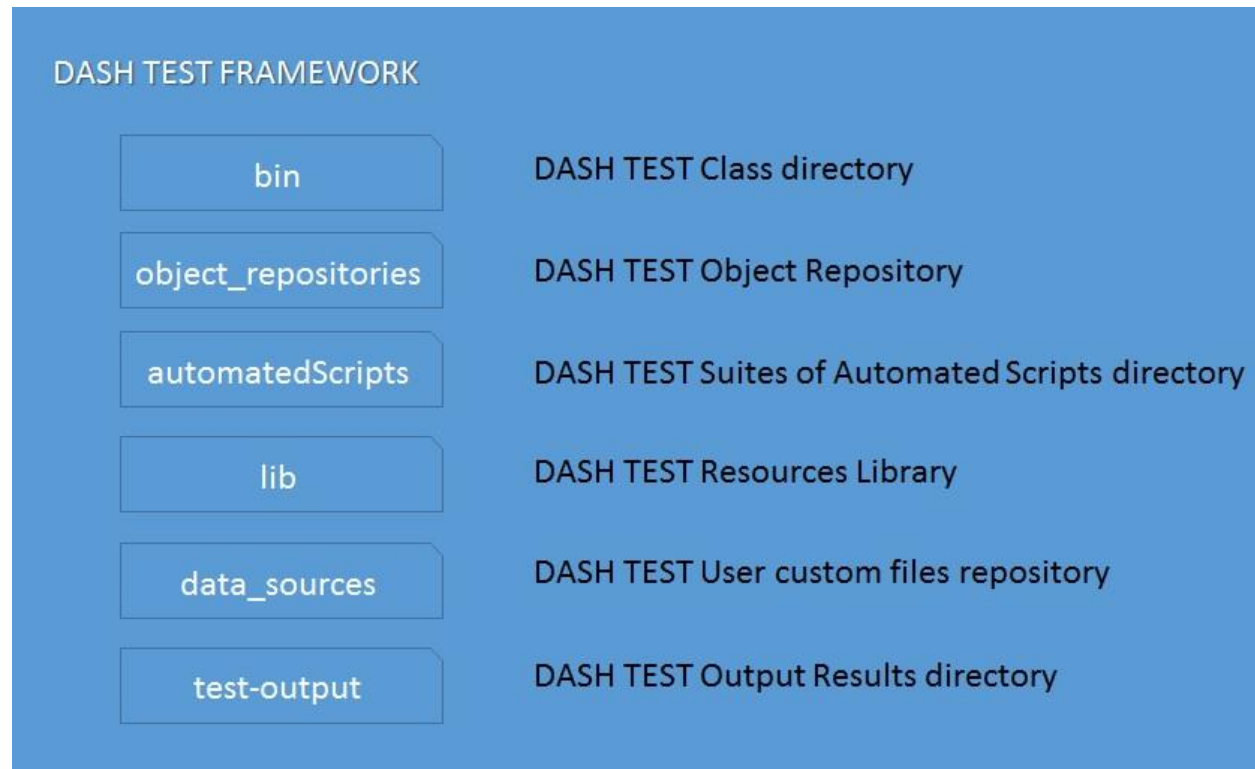
DASH TEST® – Table of Contents

Introduction.....	2
THE DASH TEST® FRAMEWORK.....	7
THE DASH TEST® Object Repository.	8
THE DASH TEST® Suite Configuration Files	9
THE DASH TEST® UI – XML Builder	10
Test Parameters Table and Test Parameters Table Controls:	12
Creating an XML Configuration file using the DASH TEST® UI.....	13
The Execute DASH_TEST button.....	20
Open an existing XML Configuration file.....	21
DASH in HEADLESS CHROME	25
DASH TEST® Suite files.....	26
Control Columns Description.	26
Parameter Columns Description.	27
Object Reference Types.	27
Automated Script Example.....	30
The DASH TEST® Component Library.	40
Action Components.....	43
ACTION KEYS	43
CLEAR_TEXT.....	43
CLICK_ALERT_IF_PRESENT	43
CLICK_COMPONENT.....	44
CLICK_COMPONENT_BY_PARTIAL_TEXT.....	44
CLICK_COMPONENT_BY_TEXT	45
CLICK_COMPONENT_IF_PRESENT.....	45
CLICK_COMPONENT_IF_STATUS.....	46
CLICK_COMPONENT_IF_TEXT	46
CLICK_COMPONENT_TO_STATUS.....	47
CLOSE_BROWSER	47
CLOSE_CHILD_TAB	48
COUNT_ELEMENTS_IN_LIST.....	48
COUNT_OPEN_TABS	49
FOCUS_ON_COMPONENT.....	49
HOVER_COMPONENT	50
HOVER_COMPONENT_BY_TEXT	50

LOCATE_IN_COMBO.....	51
LOG_ENTRY	51
LOG_STORED_TEXT	51
LOG_STORED_NUMBER	52
NAVIGATE_TO	52
PERFORM_OPERATION	53
PRESS_KEYS	53
RETURN_COMPONENT_HTML_CLASS_NAME	54
RETURN_COMPONENT_HTML_PROPERTY	54
SELECT_COMBO	55
SELECT_COMBO_BY_INDEX	55
SELECT_COMBO_BY_TEXT	56
SELECT_COMBO_BY_VALUE.....	56
SELECT_COMPONENT	57
SET_NUMBER	58
SET_TEXT	58
SET_TEXT_ENCLOSED_BY_AUX	59
SET_TEXT_NO_CLEAR.....	60
SET_WINDOW_SIZE.....	60
STORE_INPUT/STORE_INPUT_01 / STORE_INPUT_02	61
STORE_PARAMETER_NUMBER/STORE_PARAMETER_NUMBER01/STORE_PARAMETER_NUMBER02	62
STORE_PARAMETER_TEXT	62
STORE_NUMBER	63
STORE_TEXT	63
STORE_TEXT_IF_COMPONENT_PRESENT	64
SWITCH_TO_CHILD_TAB	64
SWITCH_TO_PARENT_TAB	65
TAKE_SCREENSHOT	65
USE_STORED_TEXT.....	66
USE_STORED_NUMBER.....	67
Verification Components.	68
VERIFY_COMPONENT.....	68
VERIFY_COMPONENT_BY_TEXT	68
VERIFY_COMPONENT_BY_PARTIAL_TEXT	68
VERIFY_COMPONENT_CONTAINS_TEXT	69

VERIFY_IF_COMPONENT_STATUS_IS.....	69
Wait Components.....	71
WAIT_REFRESH.....	71
WAIT_UNTIL_TEXT_IS_EQUAL_TO.....	71
WAIT_WHILE_TEXT_IS_EQUAL_TO.....	72
Assert Components.....	73
ASSERT_COMPONENT_IS_SELECTED.....	73
ASSERT_COMPONENT_IS_PRESENT.....	73
ASSERT_COMPONENT_TEXT_FEATURE.....	74
ASSERT_CONTAINS.....	74
ASSERT_CONTAINS_IF_COMPONENT_PRESENT.....	75
ASSERT_CONTAINS_IF_COMPONENT_PRESENT_AND_NOT_AUX_TEXT.....	75
ASSERT_CONTAINS_INPUT.....	76
ASSERT_CONTAINS_STORED_TEXT.....	77
ASSERT_CONTAINS_STORED_NUMBER.....	77
ASSERT_EQUALS.....	78
ASSERT_EQUALS_IF_COMPONENT_PRESENT.....	78
ASSERT_EQUALS_IF_COMPONENT_PRESENT_AND_NOT_AUX_TEXT.....	79
ASSERT_EQUALS_INPUT.....	80
ASSERT_EQUALS_NULL.....	80
ASSERT_EQUALS_TO_STORED_TEXT.....	81
ASSERT_EQUALS_TO_STORED_NUMBER.....	81
ASSERT_HTML_ATTRIBUTE_CONTAINS_PARAMETER.....	82
ASSERT_HTML_ATTRIBUTE_EQUALS_PARAMETER.....	83
ASSERT_NOT_CONTAINS.....	83
ASSERT_STARTS_WITH.....	84
The DASH TEST© XML Suite Configuration File.....	85
Parameters passed to DASH TEST©:.....	85
Execute DASH TEST© from command prompt.....	88
Understanding “DASH TEST©” test Reports.....	89
The Emailable Report.....	90
The Full Report.....	91

THE DASH TEST® FRAMEWORK



The DASH TEST® framework consists of the following elements.

- **The DASH TEST®, Class directory.** This folder contains the java.class modules and executables that make the DASH TEST® run.
- **The DASH TEST®, Object Repository.** This folder contains the files with the definition of the objects which DASH TEST® will interact during runtime.
- **The DASH TEST®, Suites of Automated Scripts.** This folder contains the files with the steps and instructions the user wants DASH TEST® to perform on the AUT.
- **The DASH TEST®, Resources Library.** This folder contains the third-party libraries that help DASH TEST® perform several actions such as loading the parameter file and interact with the AUT. The libraries contained here are Selenium, Test-NG, Apache-POI and JAVA Common Language libraries. In addition, the webDriver libraries for executing tests in Google Chrome, Microsoft Internet Explorer web browsers and the icon image files for DASH TEST®.
- **The DASH TEST® User custom files.** This is an open directory to store sample text, image or document files that can be used during automated test execution.
- **The DASH TEST®, test Output directory.** This folder contains the test logs, test results and screenshots generated during execution. Log files are generated with the assistance of the Test-NG framework.

THE DASH TEST[®] Object Repository.

The object repository, is collection of text flat files, named object libraries. Each object library contains property identifications of those objects with which DASH TEST[®] will interact during execution time.

There are some rules to build an object library.

Each object library must have a unique name, with no spaces and ending with the “.properties” extension. Example: “*loginScreen.properties*”.

The object library contains couples of objectName=propertyIdentification combos, declared in a single line. Each item must be unique, to avoid DASH TEST[®] interacting with the incorrect component in the AUT.

The Object Name must not contain spaces, punctuation or special symbols. Numbers, dashes, underscore, uppercase and lowercase letters are allowed.

The Object Name uses the lowerCamelCase multiword name syntax for naming object references, with the first letter lowercase and the first letters of subsequent words in uppercase.

It is recommendable that, for all your Object Names, add a termination that reminds you what type of locator method will be used on the “propertyIdentification” segment. So then, you can make a correct reference in your script file.

- **Id** for locator method by id. Example: submitButtonId.
- **Nm** for locator method by Name. Example: cancelButtonNm.
- **Xp** for locator method by XPath. Example: warningMessageLabelXp.

It is possible to enter object references using the XPath 1.0 locator formulas, such as “*starts-with*” and “*contains*”. The use of the function “*ends-with*” is discouraged, not all Web Browsers support it, as this is an XPath 2.0 feature.

ObjectName and PropertyIdentification must be separated with the “=” operator.

These are examples of valid objectName=propertyIdentification combos:

```
cJuvenileReportFormTrtRptFamRD1_Old=cJuvenileReportForm:trtRptFamRD1:0  
searchMyReferralsFormvoucherConfNumNm=myReferralsForm:voucherConfNum  
myReferralsFormVoucherListFirstEntryXp=//*[@id='myReferralsForm:voucherList:0:getVoucherDetail']  
voucherOfficerInList227Xp=//*[@id='createVoucherForm:officer_panel']/div/ul/li[228]  
usernameXp=//input[contains(@name,'User')]  
passwordXp=//input[starts-with(@type,'pass')]
```

As a general recommendation, group your object definition by application screen, form or module. This simplifies locating and identifying each entry.

THE DASH TEST[®] Suite Configuration Files

DASH TEST[®] uses Test Suite configuration files to obtain the parameters needed to execute actions and interact with the component on a Web Page.

The Test Suite configuration files use XML notation syntax, based on the TestNG model.

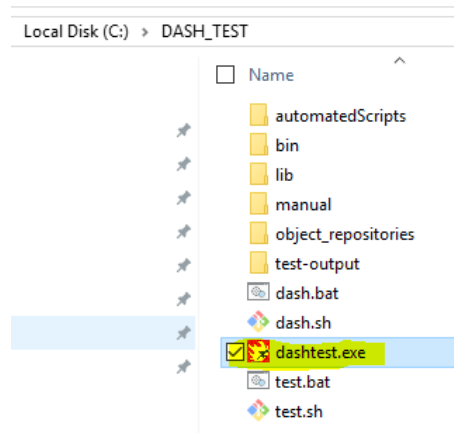
This is an example of a typical XML Test Suite configuration file for DASH TEST[®]:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
<suite name="testSuite">
<!-- This is a comment -->
  <test verbose= "2" name="Test Number 1">
    <parameter name="webBrowser" value="CHROME"/>
    <parameter name="scenarioFilePath" value="ScriptFile.xls"/>
    <parameter name="testCaseID" value="WorkSheetName"/>
    <parameter name="testCaseName" value="Test Case Title"/>
    <parameter name="objectFilePath" value="objectLibrary.properties"/>
  <classes>
    <class name="dASH_Main.DASH"/>
  </classes>
</test>
</suite>
```

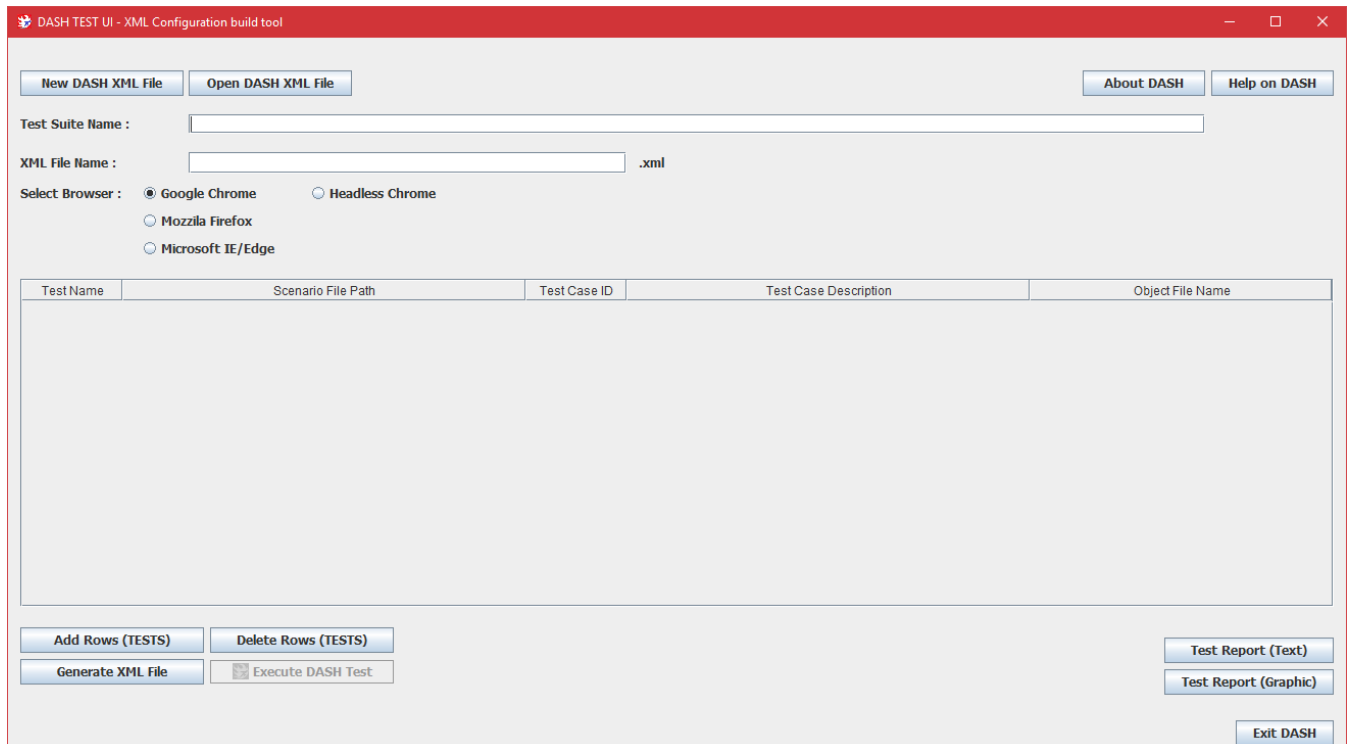
THE DASH TEST® UI – XML Builder

DASH TEST® includes a User Interface (UI) that can be used to simplify the generation of the Test Suite Configuration file.

To start the DASH TEST® UI, locate the file “*dashtest.exe*” in the DASH_TEST directory and double click on it.



The DASH TEST® UI Screen displays.



The following is a description of the DASH TEST® UI elements:

Test Suite Name: It is the name that will identify the Test Suite. This will be the name of the folder, in the test result directory, where the test result reports and screenshots will be stored. Avoid using special characters other than dashes “-” or lower bars “_”.

XML File Name: The file name where the configuration information will be stored. You don’t need to provide an extension; DASH TEST® will attach it to the name when file is generated. Be sure to ***not use “blank” spaces*** in your file name. For example, if you have typed “My Test”, use “My_Test”, “My-Test” or “MyTest” instead

Browser Selection Radio Buttons: You can select the browser that will be used to run your tests. If you want to run your tests in different web browsers, you need to create a configuration file for each browser, it is not possible to exchange browsers at the middle of a test execution.

Test Parameters Table and Test Parameters Table Controls:

The information from this table will be used in the Test Result Report. The table is divided by 5 columns.

1. **Test Name:** This column will store the Test Case Name that will be executed by DASH TEST®.
2. **Scenario File Path:** Use this column to store the path and file name to the Test Script file. Be sure that your directory names don't have any "blank" spaces in them. For example, if you have a directory named "My Files", rename it and use "My_Files", "My-Files" or "MyFiles" instead.
3. **Test Case ID:** Use this column to tell DASH TEST® what the name of the Test Sheet is, in the Test Script file, that contains the steps to execute while test execution. Be sure that the Test Case ID doesn't have any "blank" spaces in it. For example, if you have a Test Case ID named "My Test", rename it and use "My_Test", "My- Test" or "MyTest" instead.
4. **Test Case Description:** You can use this column to give a general description of what your test case does.
5. **Object File Name:** Use this column to store the path and file name to the Object Repository file. Be sure that your directory names don't have any "blank" spaces in them. For example, if you have a directory named "My Files", rename it and use "My_Files", "My-Files" or "MyFiles" instead.

Add Rows (TESTS) button: By clicking on this button, you will insert a row in the Test Parameters Table. Each row corresponds to a Test Case that DASH TEST® will execute. If you select a row, a new row will be inserted below. If no row is selected, a new row will be inserted at the bottom of the table.

Delete Rows (TESTS) button: By clicking this button, only the selected row will be deleted. If you don't select a row, nothing will happen.

Generate XML File button: When all the required information has been completed, you can press this button to generate the DASH TEST® Configuration file. Once the file has been successfully created, the Execute DASH TEST® button will display.

Execute DASH Test® button: As the button says, by clicking on it, DASH TEST® will execute using the configuration parameters contained in the current Test Suite Configuration file.

Open DASH XML File button: Opens and loads the information contained in a valid DASH TEST® Configuration file.

For more information, regarding the elements contained in the DASH TEST® Configuration file, check "What are the contents of the DASH TEST® Configuration File?" later in this guide.

New DASH XML File button: Clears the DASH TEST® screen and prepares for a new xml file. Any unsaved information in the form, will be lost.

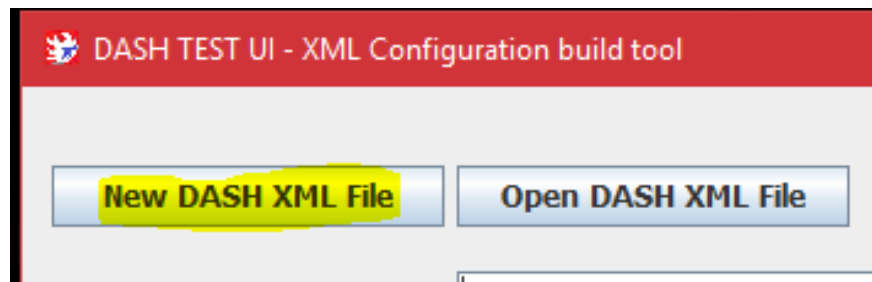
Test Report (Text) and **Test Report (Graphic)** buttons: If there is a report to display, DASH TEST® will open the latest test report in a web browser window. The web browser used is the one defined by default in your system. For more information about the report types check "Understanding "DASH TEST®" test Reports" in this manual.

Help on DASH button: Will display a digital version of this manual.

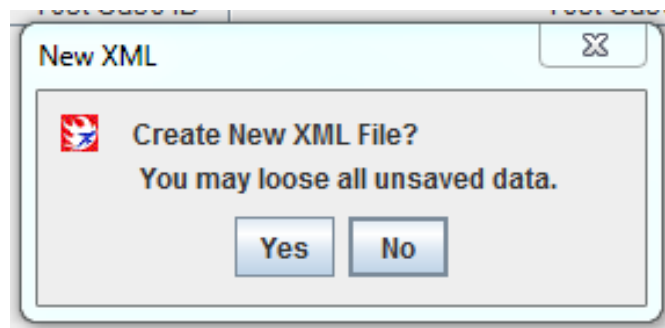
Creating an XML Configuration file using the DASH TEST[®] UI.

Open DASH UI by executing the *DASH_UI.jar* file in the DASH_TEST directory.

Press the “New DASH XML File” button.

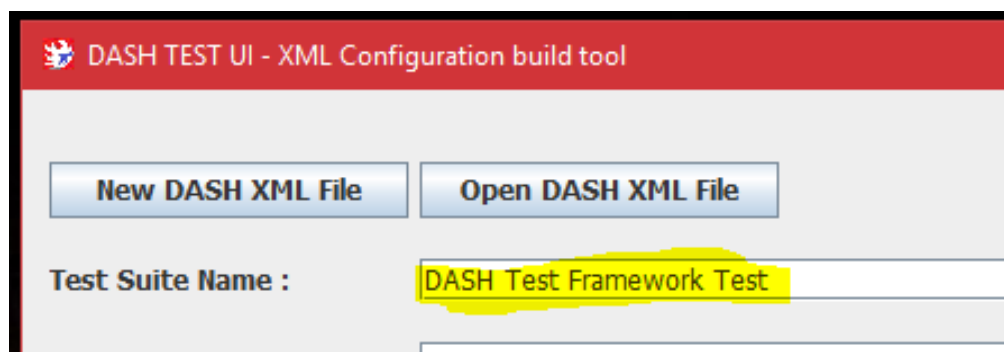


DASH_TEST shows the following dialog:



This is a warning, saying that if you have already placed data in any of the controls, the information will be lost. Since we are just starting a new DASH_TEXT XML file, press **Yes**.

In the **Suite Name** text bow, enter the name you want to give to your test suite.



Name your configuration file, by typing a file name in the File Name textbox. It is recommended to use a short descriptive name. You don't have to supply an extension, DASH will add the “xml” extension to the file name.

DASH TEST UI - XML Configuration build tool

New DASH XML File **Open DASH XML File**

Test Suite Name : DASH Test Framework Test

XML File Name : dashTst .xml

Select the browser where all tests will be executed. A test suite cannot be executed in more than one browser. If you need your test to be executed in different browsers, you need to create a Configuration File per web browser. NOTE: Your system must have installed the browser you want your tests to be executed, if your system doesn't have the selected browser, you must download and install it.

DASH TEST UI - XML Configuration build tool

New DASH XML File **Open DASH XML File**

Test Suite Name : DASH Test Framework Test

XML File Name : dashTst .xml

Select Browser :

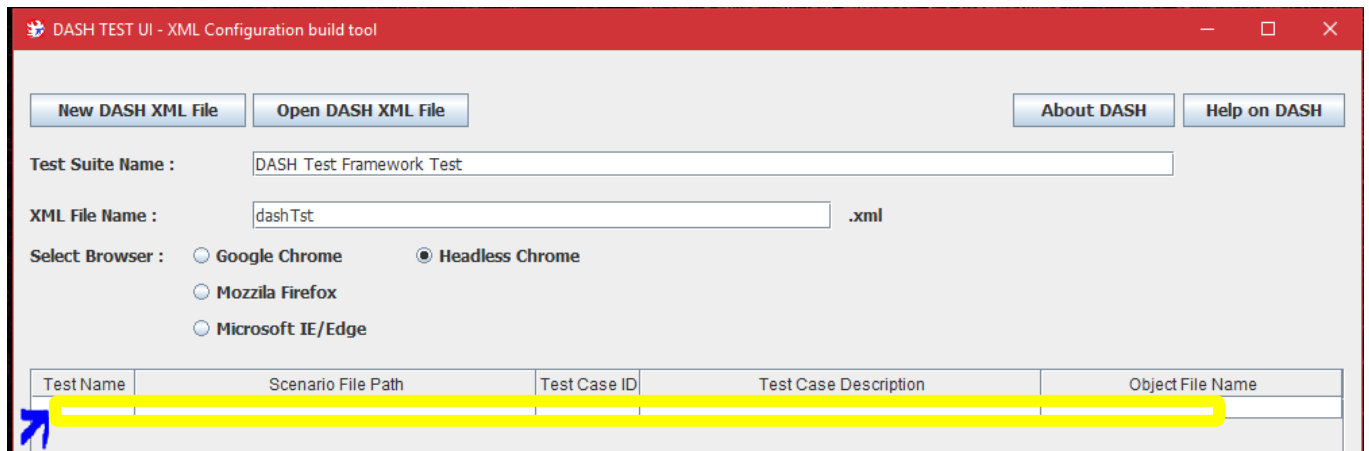
- ☒ Google Chrome
- ☐ Headless Chrome
- ☐ Mozilla Firefox
- ☐ Microsoft IE/Edge

The Test Parameters Table is empty, to add tests (rows) press the Add Rows (tests) button.

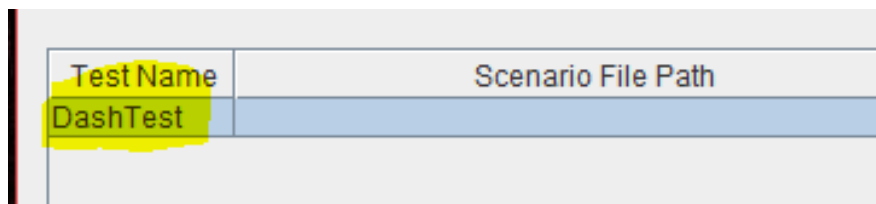
Add Rows (TESTS) **Delete Rows (TESTS)**

Generate XML File **Execute DASH Test**

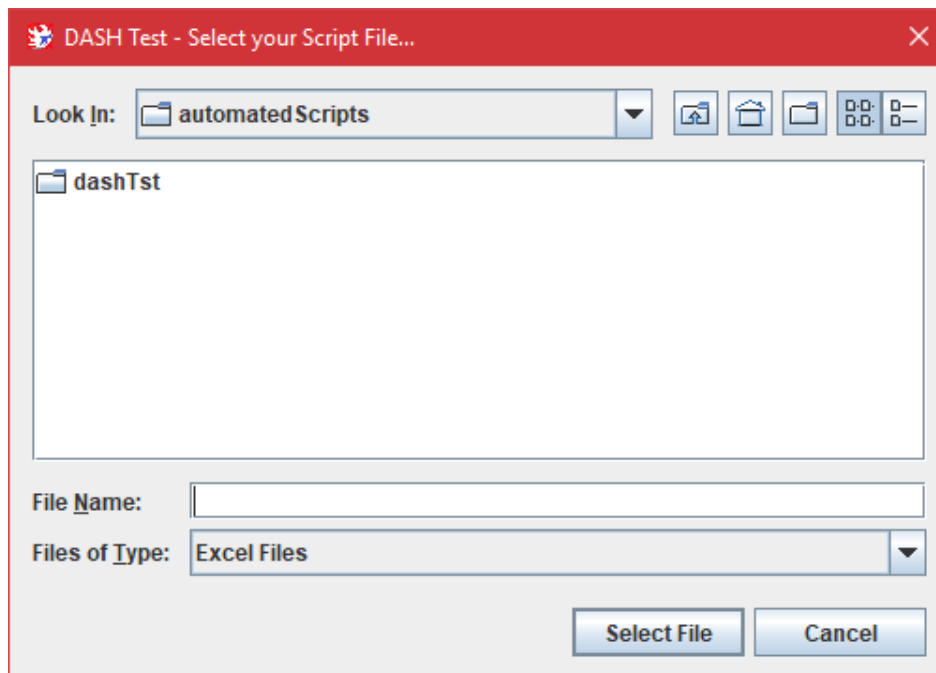
After pressing the Add Rows (tests) button, a new row is added to the table.



At the Test Name column, enter a name for the Test Case.

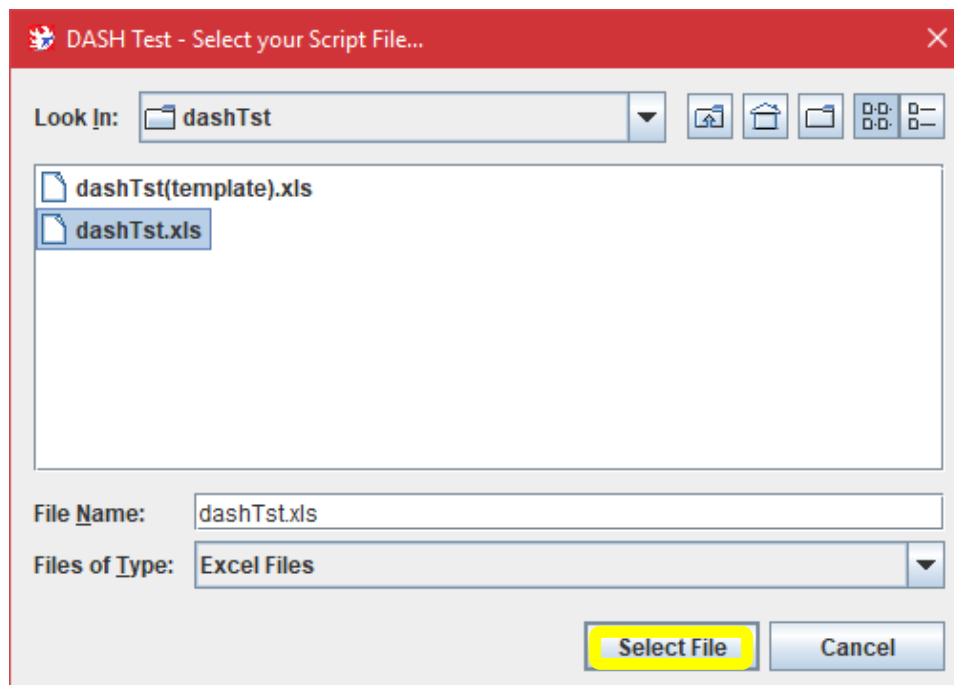


Using your mouse, click on the Scenario File Path column, the following dialog will display.



Use this dialog box, to locate your script files. It is mandatory that your script files are contained in the automatedScripts directory. For your convenience, you can place them in subfolders, but these subfolders must be located inside the automatedScripts directory. For information about how to create your DASH_TEST compatible scripts files, refer to the chapter titled “DASH TEST® Suite Files”.

Select your file and press the Select File button.



The full path to the Script file is populated in the Scenario File Path.

Test Name	Scenario File Path
DashTest	dashTst\dashTst.xls

In the Test Case ID column, enter the name of the Worksheet, in the Test Script file, where the test steps are stored.

Test Name	Scenario File Path	Test Case ID
DashTest	dashTst\dashTst.xls	DashTest

This is the ID

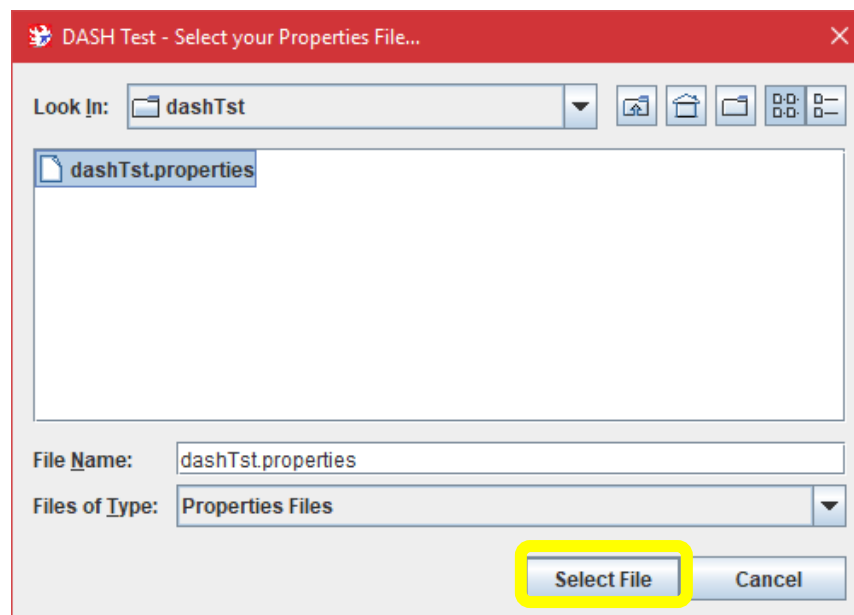
DashTest

In the Test Case Description, you can enter an open explanation about what your test case does.

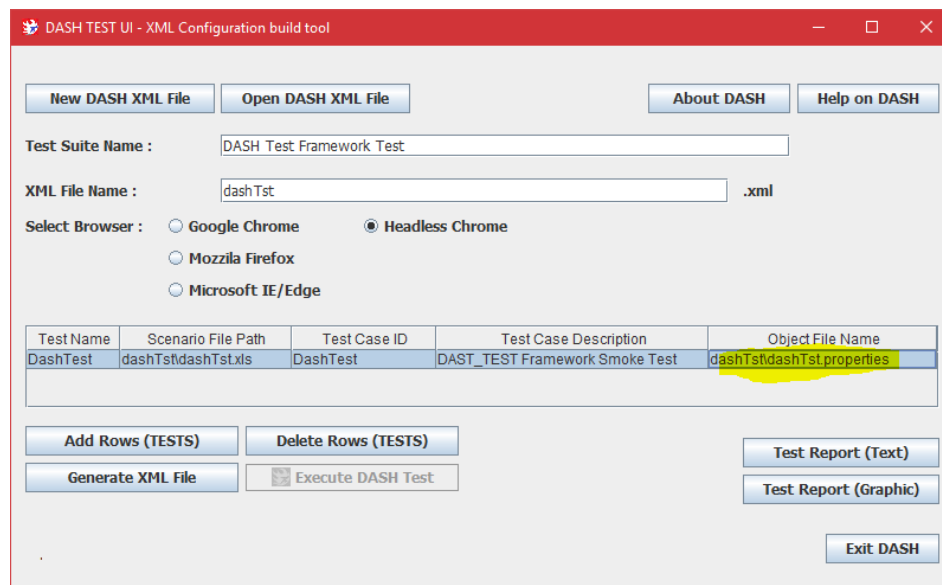
Test Name	Scenario File Path	Test Case ID	Test Case Description
DashTest	dashTst\dashTst.xls	DashTest	DAST_TEST Framework Smoke Test

Using your mouse, click on the Object File Name column. Select the object properties file that contains the reference to the objects used by your test scripts. Refer to the chapter “THE DASH TEST® Object Repository” for more information about how to build your object repository files. All your Object Files must be contained inside the object repositories directory.

Select your Object File and press the Select File button.



The object file name is populated in the Object File Name column.



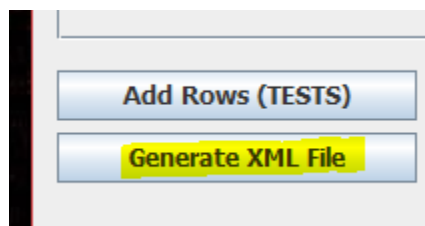
If you want to add more tests to the suite, repeat the previous steps.

In the following screenshot, I have added one more test to the Test Suite.

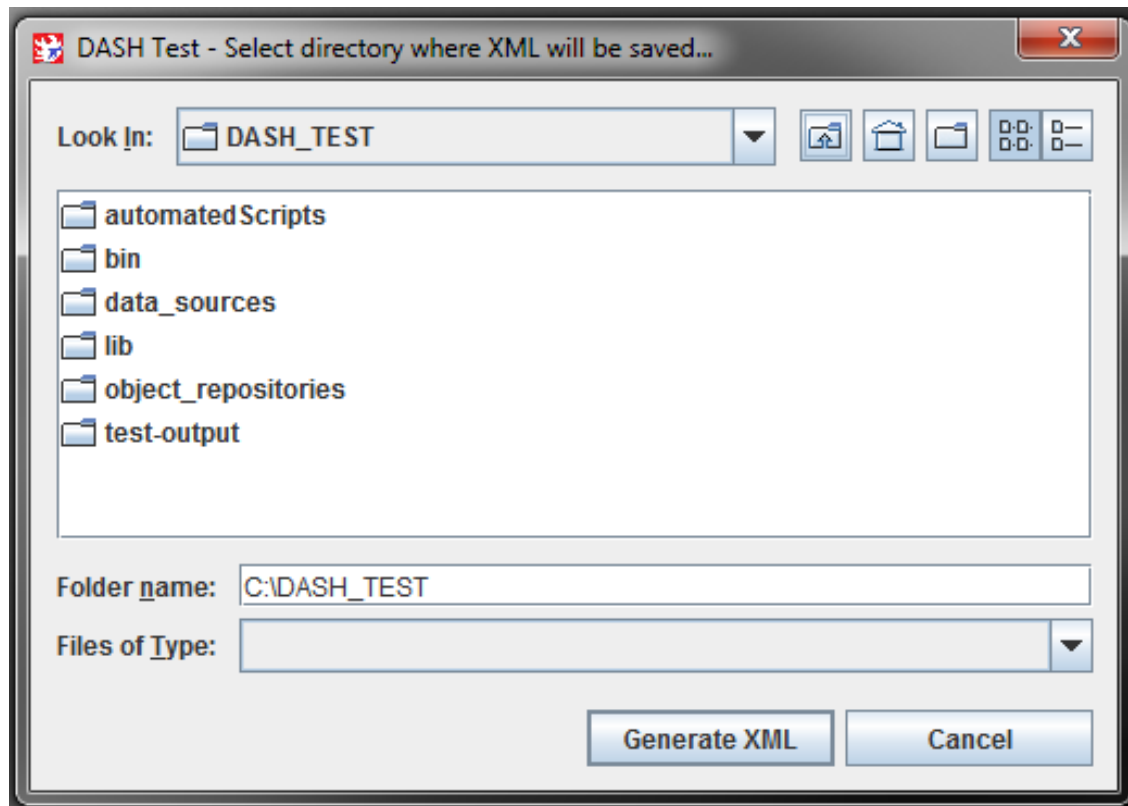
Test Name	Scenario File Path	Test Case ID	Test Case Description	Object File Name
DashTest	dashTst\dashTst.xls	DashTest	DASH TEST Framework Smoke Test	dashTst\dashTst.properties
DashTest2	dashTst\dashTst.xls	DashTest2	Test 2	dashTst\dashTst.properties

With this configuration, DASH TEST® will execute the test suite by first, running Test_01 following the steps stored in the “*TS-Angular01.xls*” script file, using the provided parameters, then running Test_02 following the steps stored in the “*TS-Angular01.xls*” script file, using the same Object File. The difference is the Test Case ID, which tells what test case will be executed.

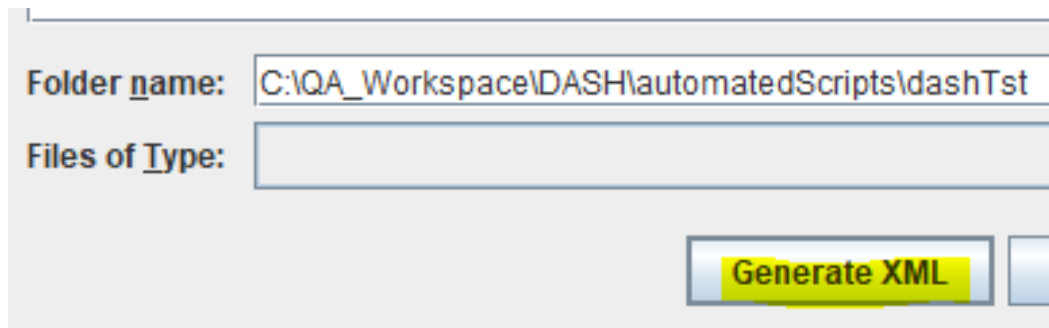
Press the Generate XML File button...



...the following dialog displays:

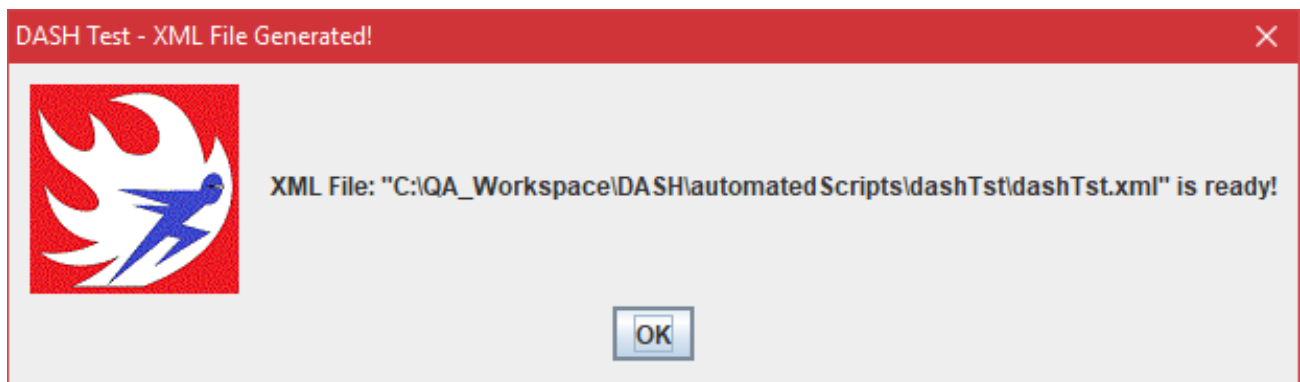


Press the **Generate XML** button, to save your configuration in an XML file, with the name you typed in the File Name field.



The screenshot shows a dialog box with two input fields. The first field is labeled 'Folder name:' and contains the text 'C:\QA_Workspace\DASH\automatedScripts\dashTst'. The second field is labeled 'Files of Type:' and is currently empty. To the right of these fields is a yellow button with the text 'Generate XML'.

Upon successful Configuration File generation, the following message will display:



Press the OK button to close the message box.

The Execute DASH_TEST button.

The “Execute DASH_TEST” will be enabled to the GUI. In case you make any changes to the configuration controls, this button will disappear, because DASH TEST® is expecting you to generate the Configuration File again, so the new configuration can be used.

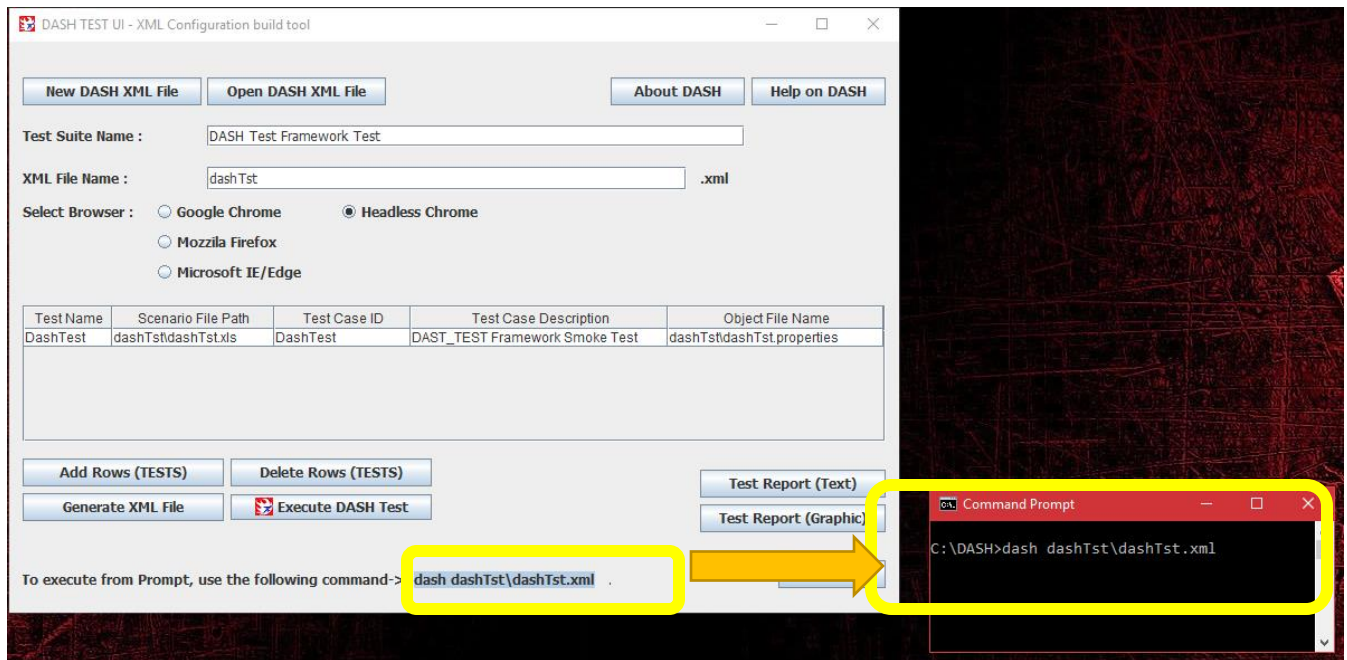
The screenshot shows the 'DASH TEST UI - XML Configuration build tool' window. It features a red title bar and a light gray background. At the top, there are four buttons: 'New DASH XML File', 'Open DASH XML File', 'About DASH', and 'Help on DASH'. Below these, there are two text input fields: 'Test Suite Name : DASH Test Framework Test' and 'XML File Name : dashTst.xml'. Underneath the XML File Name field, there are four radio buttons for 'Select Browser': 'Google Chrome', 'Headless Chrome' (which is selected), 'Mozilla Firefox', and 'Microsoft IE/Edge'. A table with five columns is present: 'Test Name', 'Scenario File Path', 'Test Case ID', 'Test Case Description', and 'Object File Name'. The table contains one row with the following data: 'DashTest', 'dashTst\dashTst.xls', 'DashTest', 'DAST_TEST Framework Smoke Test', and 'dashTst\dashTst.properties'. Below the table, there are four buttons: 'Add Rows (TESTS)', 'Delete Rows (TESTS)', 'Generate XML File', and 'Execute DASH Test' (which has a red icon). To the right of these buttons are two more buttons: 'Test Report (Text)' and 'Test Report (Graphic)'. At the bottom, there is a text area with the command 'To execute from Prompt, use the following command-> dash dashTst\dashTst.xml .' and an 'Exit DASH' button.

Test Name	Scenario File Path	Test Case ID	Test Case Description	Object File Name
DashTest	dashTst\dashTst.xls	DashTest	DAST_TEST Framework Smoke Test	dashTst\dashTst.properties

For Windows users, dash has the option to execute test cases from command prompt. Under the “Execute DASH_TEST” button, DASH let you know the syntax you can use to do it.

This image is a close-up of the 'Execute DASH Test' button and the command prompt instruction. The button is blue with a red icon and the text 'Execute DASH Test'. Below it, the text 'To execute from Prompt, use the following command-> dash dashTst\dashTst.xml .' is highlighted in yellow.

In this case at the command prompt, simply type the following statement...

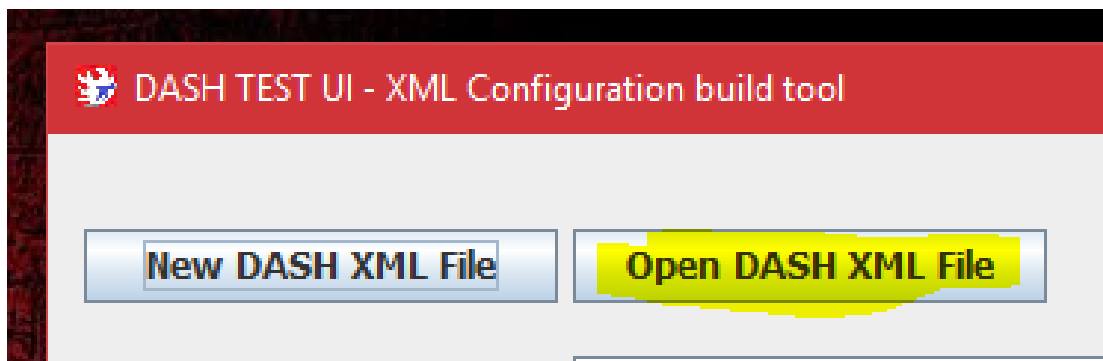


For more information about how to execute DASH TEST® in the different Operating Systems or Environments supported by DASH TEST®, refer to the chapter titled “Execute DASH TEST® from command prompt”.

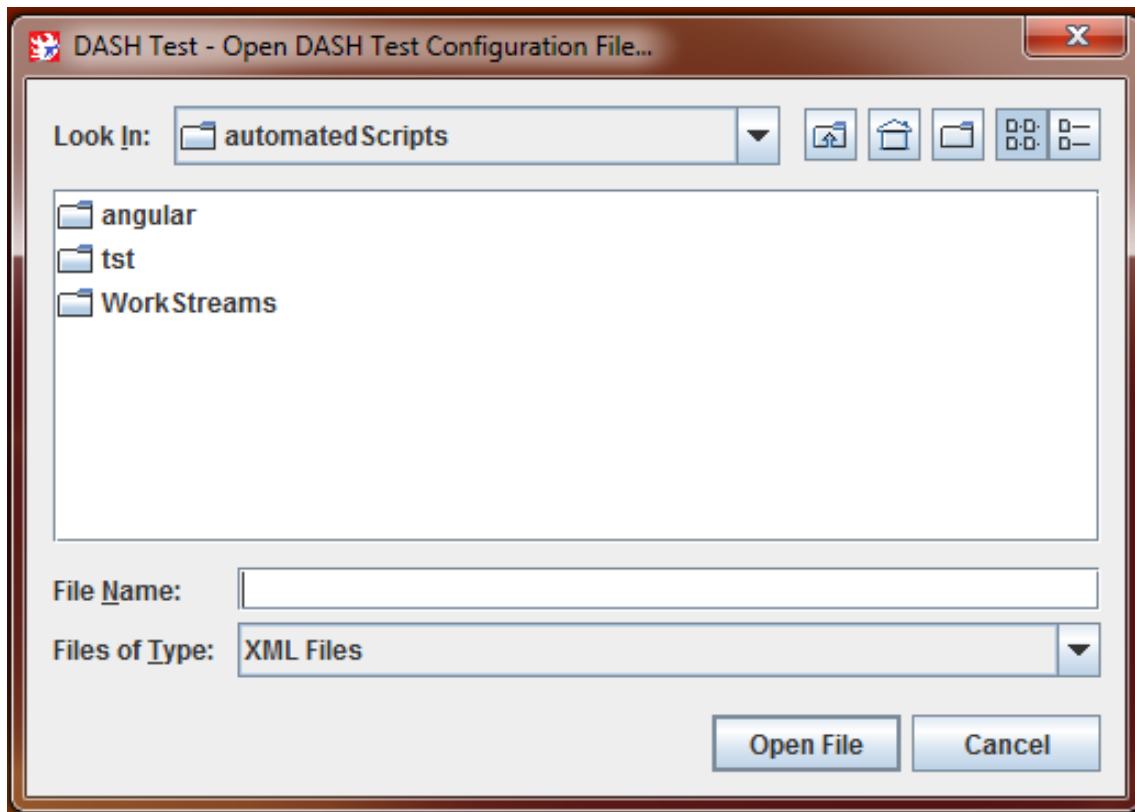
Open an existing XML Configuration file

Note: Opening an XML Configuration File, will lose any unsaved information on the DASH TEST® screen.

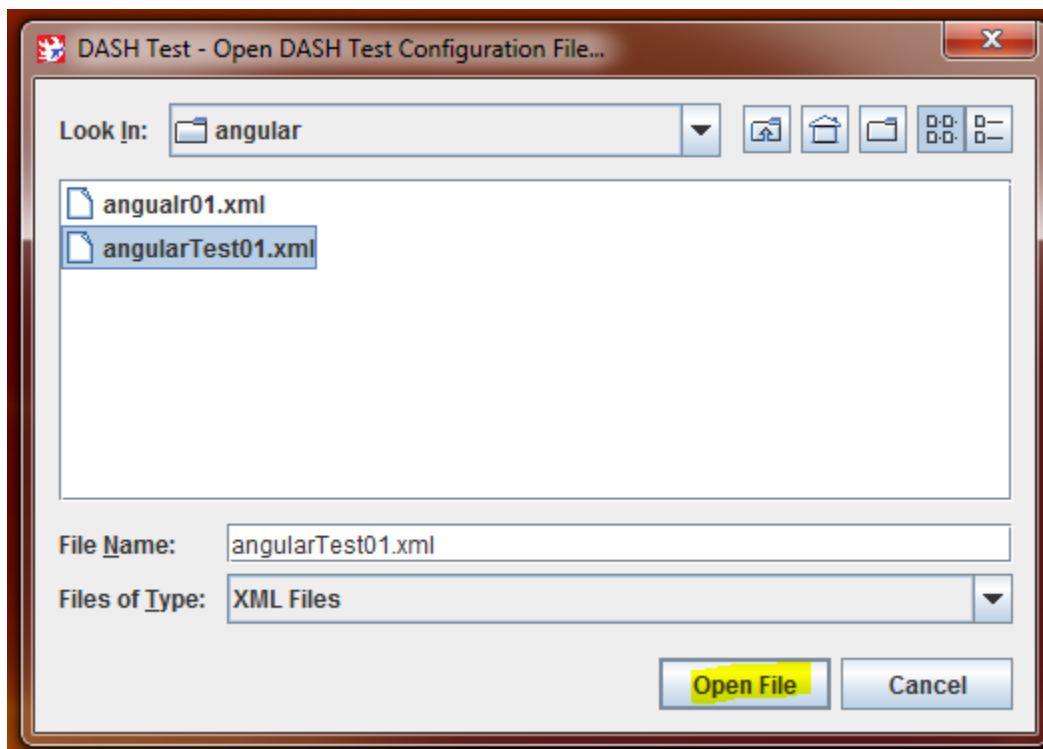
At the DASH TEST® UI screen, press the Open DASH XML File.



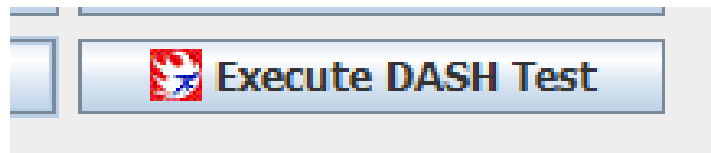
The following dialog displays:



Locate your DASH Configuration file and press the Open File button.



To execute the test suite, press the "Execute DASH_TEST" button.



DASH TEST® Executes...

```
C:\Windows\system32\cmd.exe
[TestNG] Running:
  C:\DASH_TEST\automatedScripts\angular\angularTest01.xml

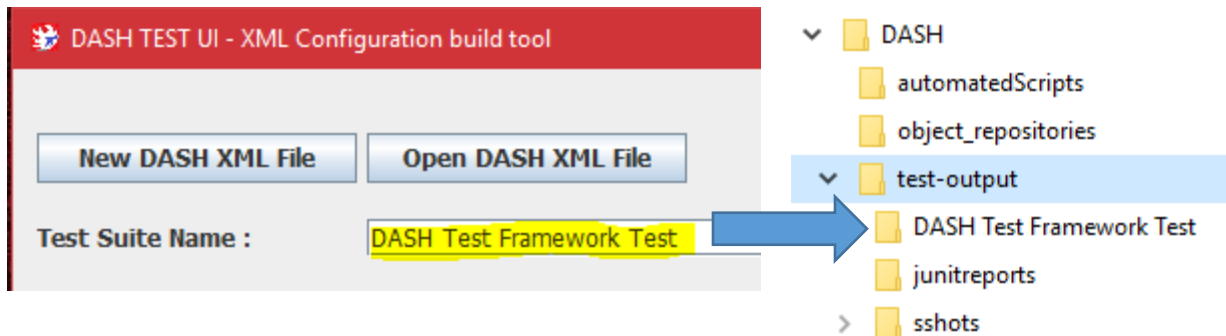
Starting ChromeDriver 2.20.353145 (343b531d31eeb933ec778dbcf7081628a1396067) on
port 46539
Only local connections are allowed.
DASH is a Copyright property of:
Edgar Chavez 2015 (PRE000008332)...
Designed and Developed by Edgar Chavez.
DASH TEST... DASHING!
Loading CHROME

*****
Executing Test Case (TCAngular01):

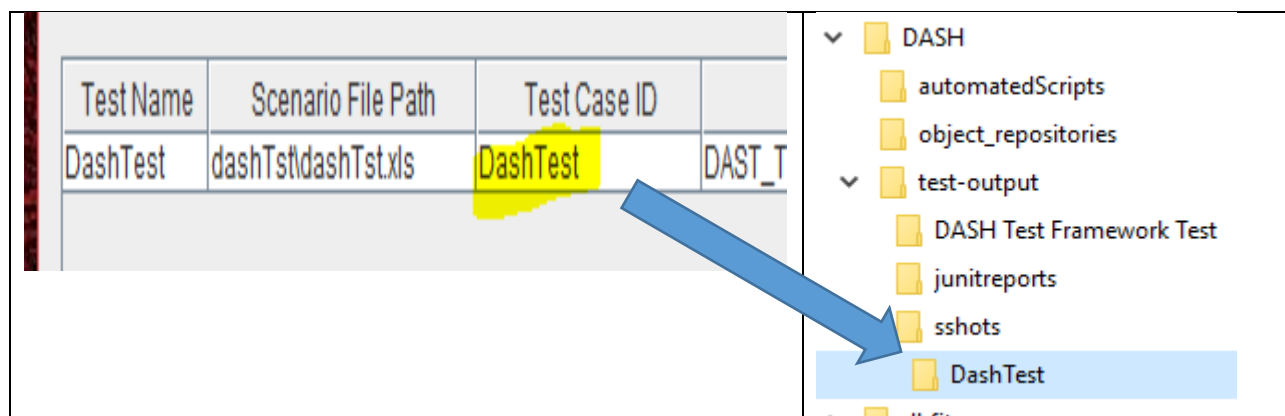
"This test case will test an Angular made Web Page"

=====
Starting Iteration #1
=====
Step Number 1a: Navigate to URL
Navigate to ->
http://body-browser.org/
=====
Step Number 1b: Verify Page Loaded
=====
Step Number 2a: Press the Mouse button
Clicked on: Mouse
=====
Step Number 2b: Verify Page Loaded
=====
Step Number 3a: Click on the "Adrenal Gland" button
Clicked on: adrenal gland
=====
Step Number 3b: Wait for result to load
=====
```

Once execution is complete, you will find your test results in the test_output directory, under the folder with the name you typed in the Suite Name field:



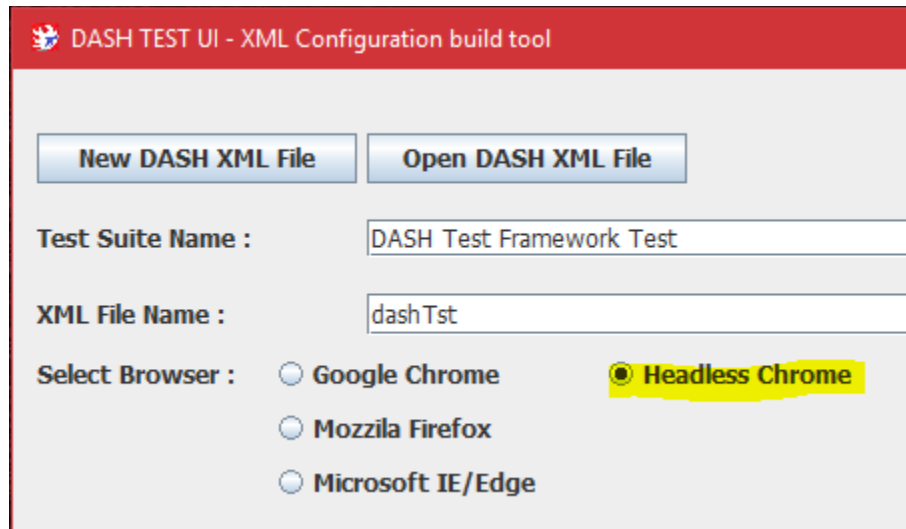
If your script includes a step for taking screenshots, those are in the sshots directory, under a folder named with the Test Case ID.



For more information about the test results, read the chapter titled “Understanding DASH_Test test Reports”.

DASH in HEADLESS CHROME

Dash also allows the option to execute test in Chrome Headless mode. This is only available for Google Chrome Version 59+. NOTE: Be sure that your version of Chrome Driver and Google Chrome are both up to date and are compatibles.



The screenshot shows the 'DASH TEST UI - XML Configuration build tool' window. It features a red title bar with a small icon on the left. Below the title bar, there are two buttons: 'New DASH XML File' and 'Open DASH XML File'. Below these buttons, there are two text input fields. The first is labeled 'Test Suite Name :' and contains the text 'DASH Test Framework Test'. The second is labeled 'XML File Name :' and contains the text 'dashTst'. Below the input fields, there is a section labeled 'Select Browser :'. It contains three radio button options: 'Google Chrome', 'Mozzila Firefox', and 'Microsoft IE/Edge'. The 'Headless Chrome' option is highlighted with a yellow background, and it has a selected radio button.

DASH TEST UI - XML Configuration build tool

New DASH XML File **Open DASH XML File**

Test Suite Name : DASH Test Framework Test

XML File Name : dashTst

Select Browser :

- ☐ Google Chrome
- ☒ **Headless Chrome**
- ☐ Mozzila Firefox
- ☐ Microsoft IE/Edge

DASH TEST® Suite files.

DASH TEST® Suite files are Microsoft Excel Workbooks, where the scripts containing the steps to execute are stored. A Test Suite file represents a Test Scenario. Each file can contain several worksheets or Test Case Worksheet.

Each Test Case Worksheet consists of a header row and a series of data rows divided in 8 columns containing parameters that DASH TEST® will read and interpret, to interact with the AUT.

NOTE: All values entered in the cells MUST be entered as text or as number. DASH TEST® will convert formats such as dates currency or percentage to double numbers. DASH TEST® doesn't interpret formulas and will return an error.

If you want to enter a date as parameter, you can use the single quote (') notation in Excel to convert it into a text.

Below is an example of a Test Case Worksheet.

Ex_0	SN_1	SD_2	AP_3	OR_4	OTe_5	CX_6	CP_01	CP_02	CP
N		0				DashTest	urlTest01URL	urlTest01URL	url
Y	1a	Navigate to URL	NAVIGATE_TO						
Y	1b	Verify Page Loaded	VERIFY_COMPONENT	googleLogoID	id				
Y	2	Enter Text to Search	SET_TEXT	googleQueryInputBoxNM	name				
Y	3a	Press the Google Search button	PRESS_ENTER						
Y	3b	Verify Page Loaded	VERIFY_COMPONENT	googleQueryResultsStatsID	id				
Y	4a	Obtain the content of the results returned	STORE_TEXT	googleQueryResultsStatsID	id				
Y	4b	Take a Screenshot	TAKE_SCREENSHOT			LG	SS_Test4a	SS_Test4b	SS
Y	5	Close the browser	CLOSE_BROWSER						

Control Columns Description.

- **(EX) ExecutionFlag:** This column is used to tell DASH TEST® is the step contained in the row will be executed or not. Accepted values Y (Yes, execute current row) or N (No, don't execute current row).
- **(SN) StepNumber:** This column stores the step number executed. It refers to the step number in the manual test case script. This will display at the log files.
- **(SD) Step Description:** This is a long description of the script executed. This is the message that will display at the log files.
- **(AP) ActionToPerform:** This column stores the name of the COMPONENT action that DASH TEST® will execute. More details on the Chapter titled "DASH TEST® Component Library".
- **(OR) ObjectReference:** In this column DASH TEST® expects to find the reference name to an object in the Object Repository identified in the Test Suite configuration File (objectFilePath)
- **(OT) ObjectReferenceType:** DASH TEST® uses this column to find the location method used to find an object in the Web Page and interact with it. It can be one of the following:
 - id
 - name
 - css

- class
- xpath

NOTE: DASH TEST® is case sensitive.

- **(CX) ComponentAuxiliaryParameter:** Used by some of the components as an extra parameter, when two parameters are required to perform an action or locating an object in the web page.

Parameter Columns Description.

- **(CP) ComponentParameter_01(...02, ...03, ..., ...N):** These columns are used to pass a parameter value that, some objects in the Web Page, expect to interact with the user.

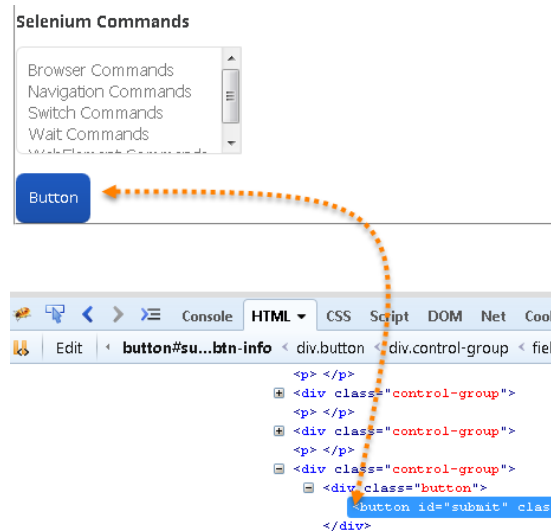
You can add as many component parameter columns as iterations of your script you want DASH TEST® to execute. The only condition is that you pass the same number of rows for every parameter column you wish to add. Note that any variation of your script, caused by passing different parameters, is an indicator that you need to create a new test case, in other words, you're testing a different functionality or feature of the AUT.

Object Reference Types.

While developing a web application and a web page, developers use several different methods to identify, locate and present information on the screen. Don't ask me why, but there seems that there is not a standardized or Best Practice for this, in the Software Programming community, some say it is time consuming others blame resource allocation or space in the server. Also, there are several Software Development styles and practices out there, so every developer will give you a different argument as to why she or he uses some Object Reference Type or none. But is our work as Software Testers to try and find out how to locate and identify the objects they place in the AUT.

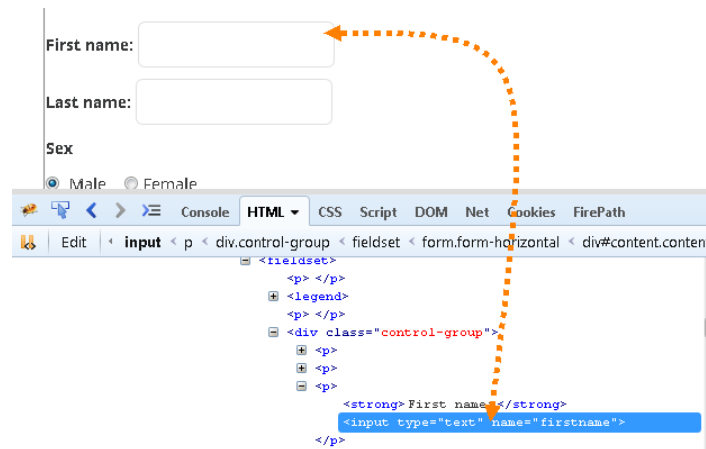
To locate those objects, there are several Object Reference Types that can be applied. Classifying those Object Reference Types using Best Practices from the most recommended to least recommended:

- **id :** This is a unique identificatory that is given to an object in the DOM of the Web Application. This Type of Reference is the most recommended and the most efficient, because it doesn't matter of the HTML or the CSS of the Web Page are modified, the object will maintain the same identification.



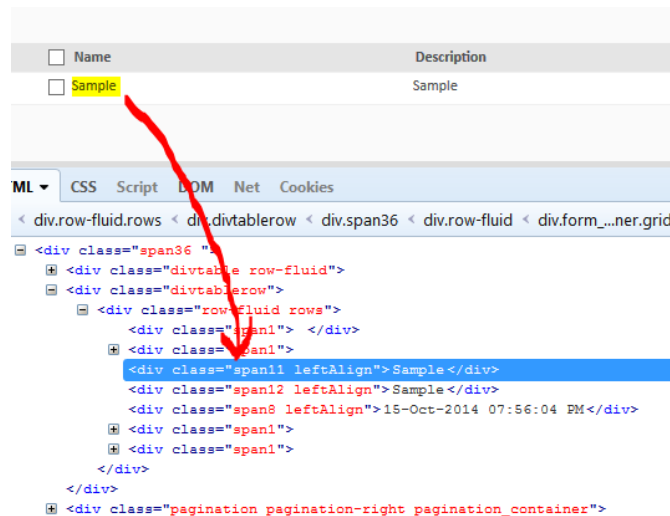
The id Object Type Reference for the object in the image is “submit”

- **name:** The objects can be given names, but it could be difficult to maintain control if another object is given the same name. Perhaps, some lamely creative developer named all his buttons, “button”.



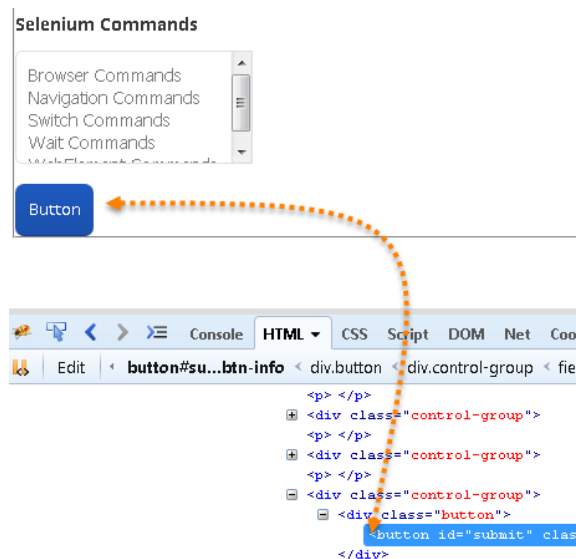
The name Object Type Reference for the object in the image is “firstname”

- **css:** The CSS controls how objects are displayed in the Web Page application. Since there are several elements that define how the object is displayed, this type contains a collection of different attributes. When using this Object Reference Type, in the Page Object Reference file, separate each of the attributes by a period “.”.



The css selector Object Type Reference for the object in the image is “div.span11.leftAlign”

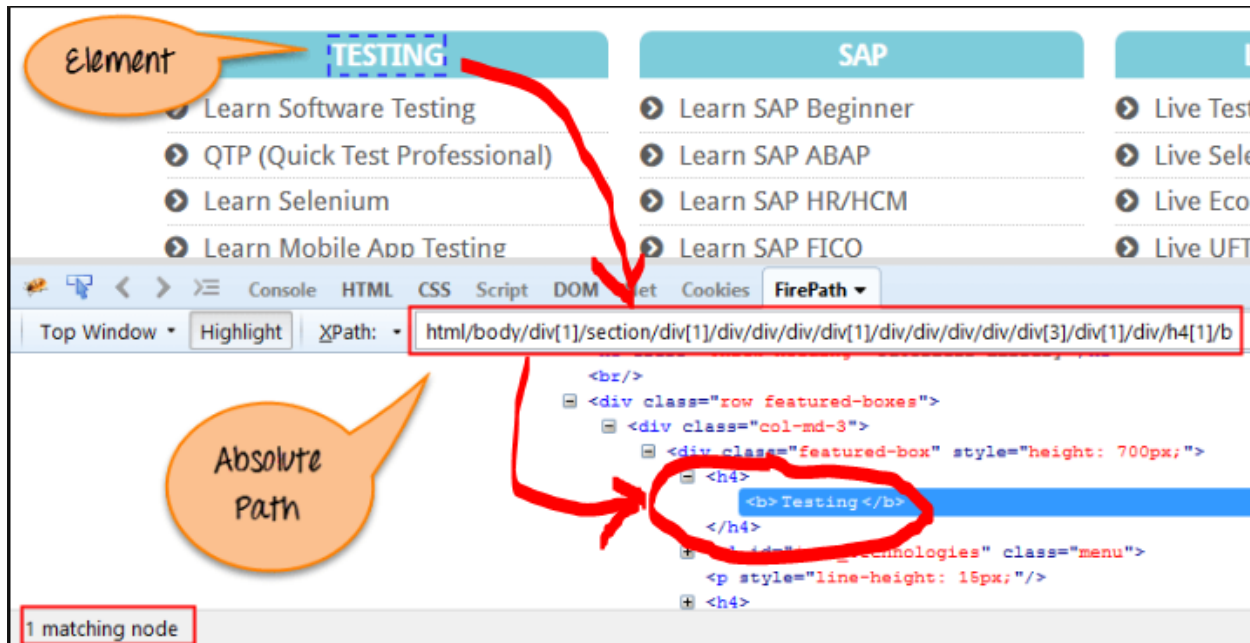
- **class:** There are different classes of objects defined in a Web Application. This applies mostly for JavaScript. When the class of an object is made of a single word, use this Object Type Reference. If it is made of several words, use the css Object Type Reference instead.



The class Object Type Reference for the object in the image is “button”

- **xpath:** The least recommended Object Type Reference. It is just a description of the absolute position of the object in the HTML and CSS code. This last resource to reference an object in the Web Application has the big limitation that, if the elements in the DOM changes, for example one object is added or removed from the page or if any modification is made to the

CSS, the probability that the xpath is affected is very high. Use this Object Type Reference as your last resource.



The xpath Object Type Reference for the object in the image is “html/body/div[1]/section/div[1]...../b”.
Can you see now why this is a terrible option?

Automated Script Example.

The following is an example of how to completely develop an Automated Test Script, that you can execute using DASH TEST© once you created the [XML Test Suite Configuration](#) file.

For the following procedure, you must:

- Have Google Chrome installed and know how to use it.
- Have internet access, have access to the google search page and know how to use it.
- Have Microsoft Excel 97 or newer installed and know how to use it.
- Have Notepad or any other simple text editor. You can use word but is not recommended.
- Have read/write access to files in your computer and know how to use it.
- Follow the instructions carefully and without skipping or altering any step.

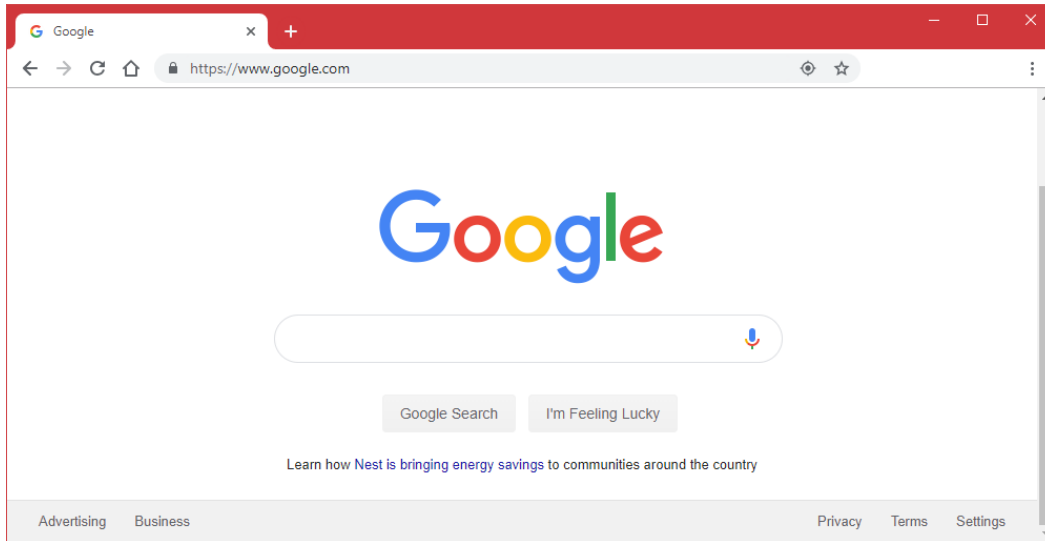
Best Practices tell that, we need a manual test script from where the Automated Test Script will be based on:

0. Open Google Chrome browser.
1. Navigate to the Google home page.
2. In the Search bar, enter search term "Man on the moon".
3. Press the Enter key to start search.
4. Obtain the number of results that matched term.

5. Take a screenshot.
6. Close the browser.

Next, “execute” the manual test script, by identifying the elements in the Web Page we will be testing.

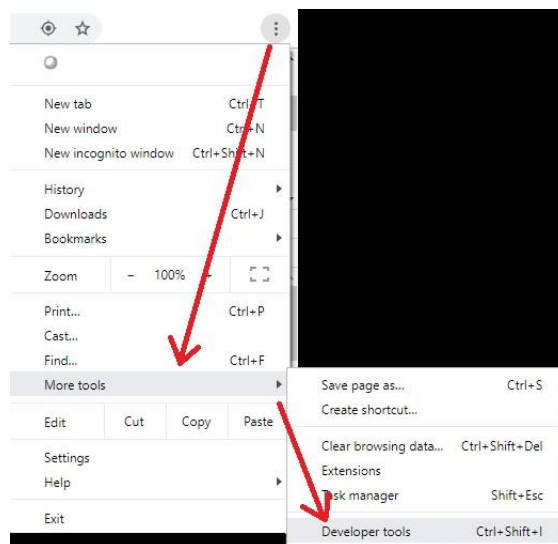
0. Open the Google Chrome Web Browser.
1. In the Address Bar enter the Google URL (www.google.com) and press enter.



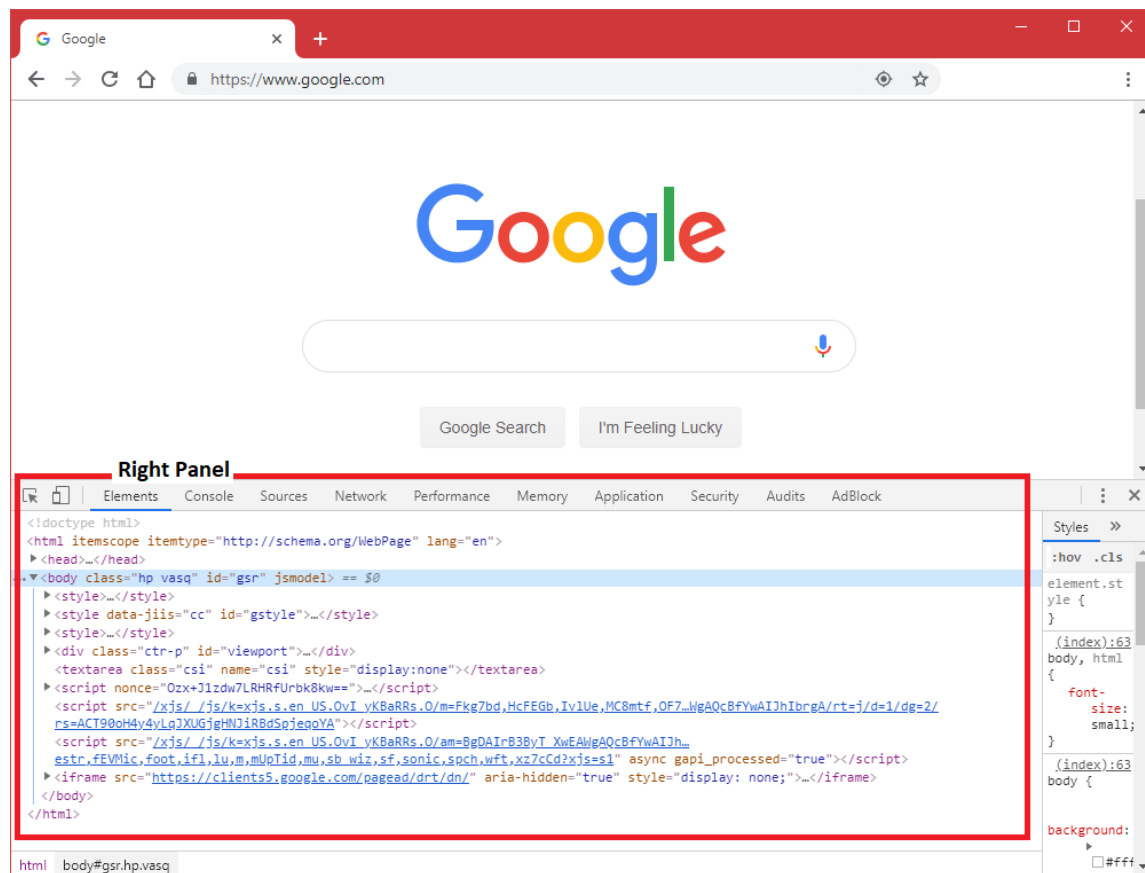
The Google home page displays. How do you know that? Of course, you can see the Google logo and the search bar. But DASH TEST© does not know that. We must identify the objects or elements in the Web Page that must be present to verify, when the Google home page is already loaded and ready to use.

For that, with your mouse click on the Google Chrome “Customize and Control” menu (the three dots on the upper right corner).

Now select the following “More tools →Developer tools”

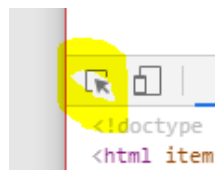


On the bottom half of the browser the Developer Tools display. We are only interested on the left side panel. That's the HTML code that contains all the visible elements in the current Web Page.



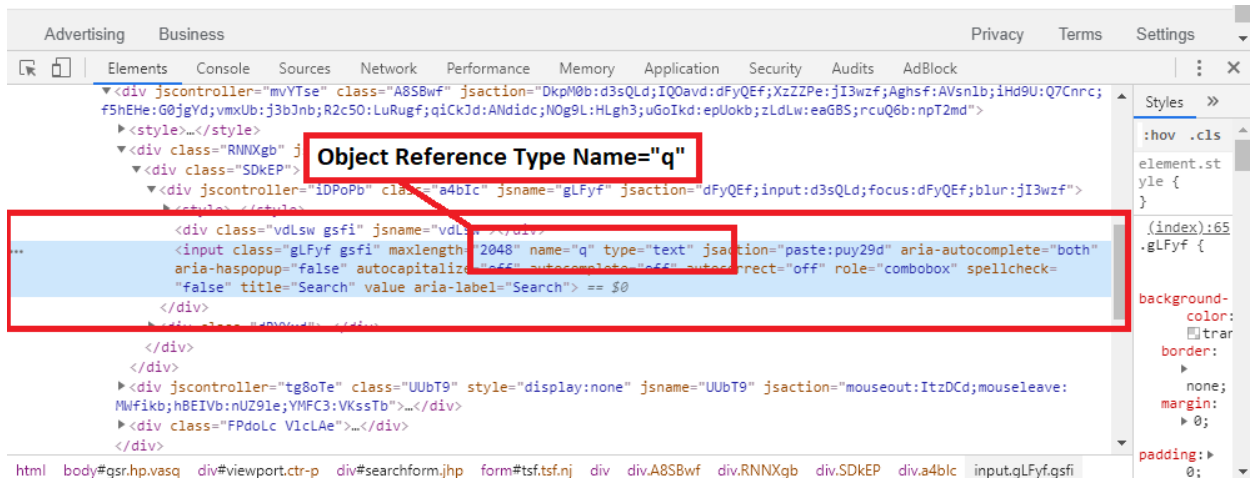
Since the action we want to perform, once the page is loaded, is enter the search term in to the Search bar, we must be sure that element is present and loaded. So, we will identify that object.

On the upper left corner of the Developer Tools panel, click on the “Inspect tool”.



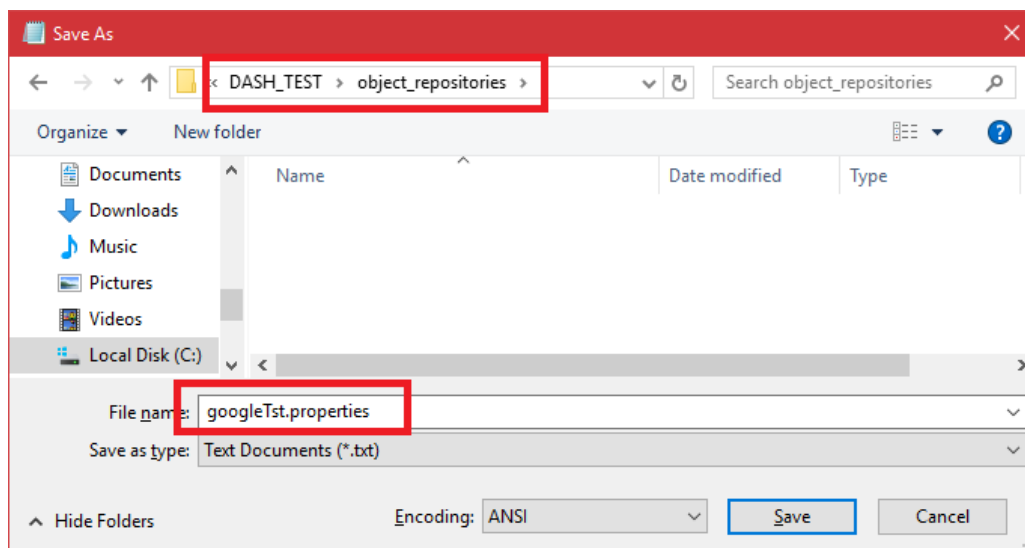
Now, click on the Search bar.

The following segment in the HTML code is selected in a blue box. All that information describes the properties and characteristics of the Search bar. We are looking for an Object Reference.

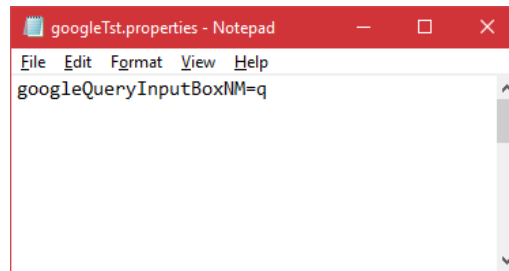


The Object Reference Type we are going to use to identify the Search bar is “name”, in the Google home page it has the value of “q”.

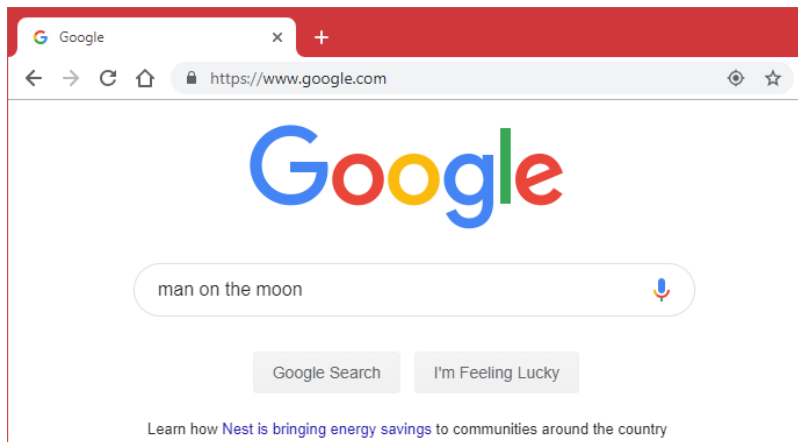
Open new text file in Windows Notepad. We are going to create a Page Object Reference file, for all the objects we are going to use in our script. Save it as “googleTst.properties” in the object_properties folder.



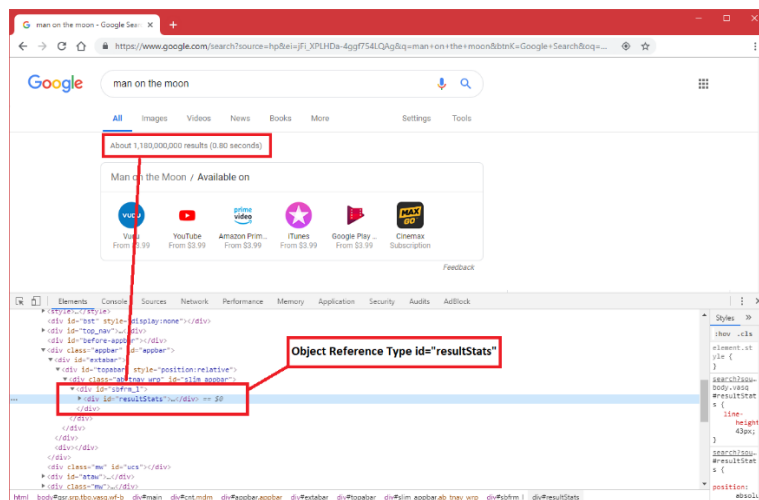
Since just “q” is not a meaningful name for the Google Search bar, we will name it googleQueryInputBox and since the Object Reference type is “name” we will call it “googleQueryInputBoxNM”. Write that in the Notepad:



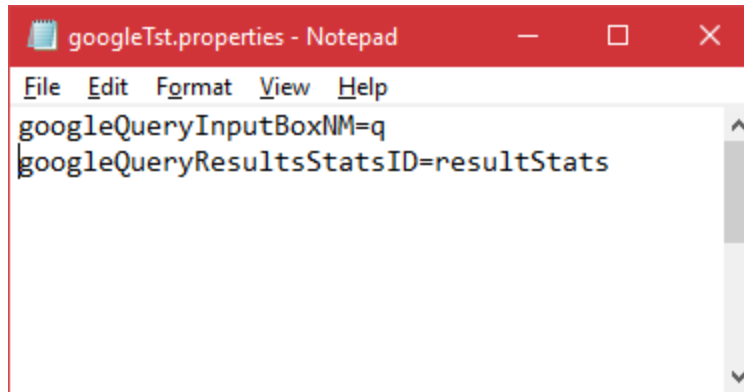
2. The next step in the script tells to enter the text “Man on the moon” on the Search bar
3. Hit the enter key in the keyboard. Do so.



4. The Search results display. Using the Inspect tool from the Developers Tool panel, inspect the text where the number of results display.



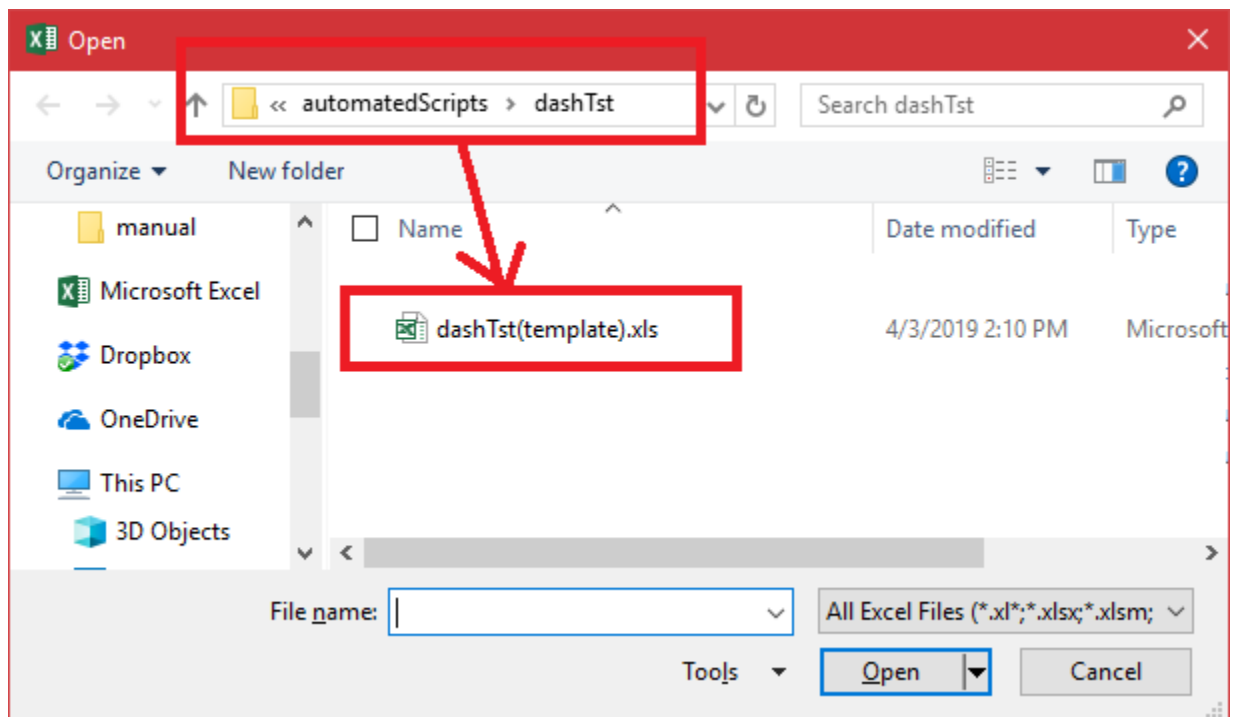
The Object Reference Type we will use is the id, with the value resultStats, again we need a more meaningful name and the Object Type Reference used, so we will call it googleQueryResultsStatsID. Add this to the Page Object Reference file:



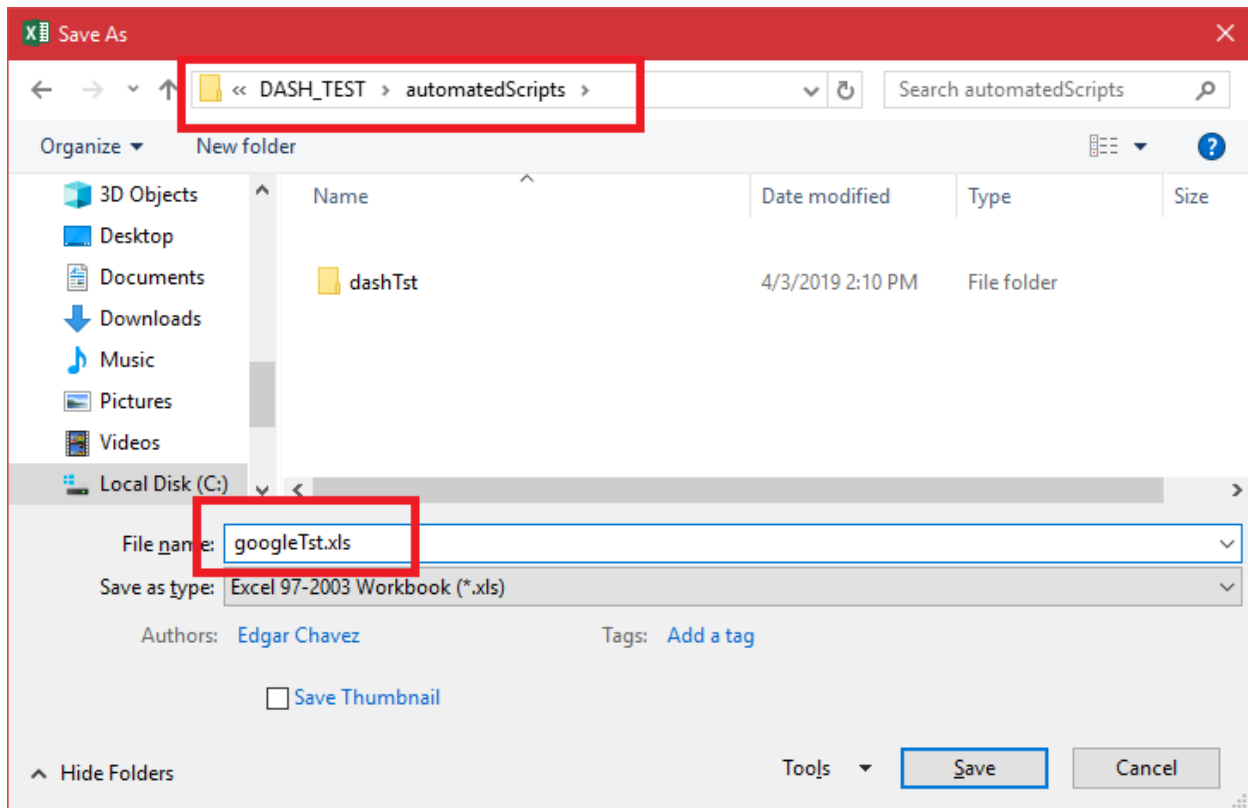
4. Then we will take a screenshot.
5. Finally, close the browser and we are done with the manual test script.

Now to the Automated Test Script.

Open your Microsoft Excel. Using the Open menu option, in the DASH TEST© automatedScripts directory, the dashTst folder. Inside is a file named dashTst(template).xls, open the file.

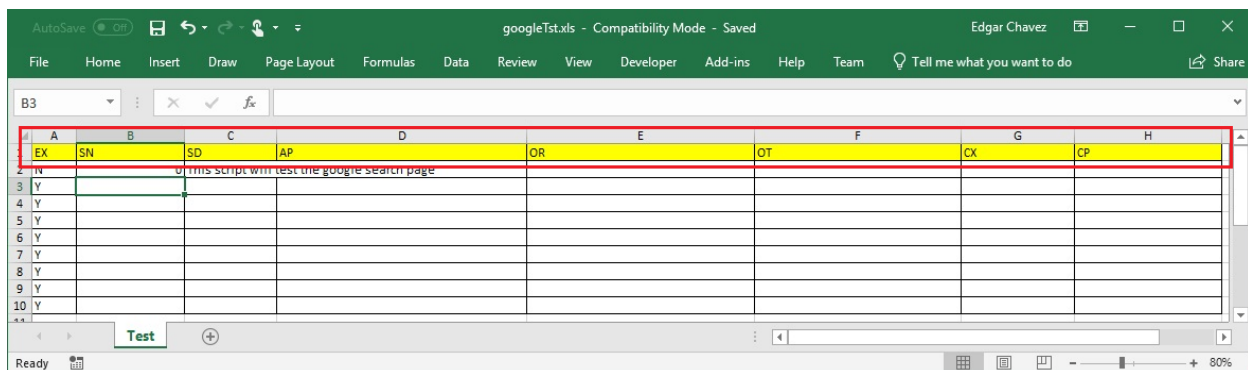


Save the file directly under the automatedScripts directory (outside the dashTst folder) as googleTst.xls



Back into the googleTst.xls Automated Test Script.

Leave Worksheet Row 1 as is. This row contains the headers that identify the information passed to DASH TEST© during execution.



At Row 2, since the Execution Flag (EX) column contains a N, meaning that this row will not execute, you can use this row for comments about prerequisites or any other important information to execute the script.

The script will start on row number 3. EX column must have a Y, meaning that this step WILL BE EXECUTED.

- Step 0 is “Open Google Chrome browser”, but since DASH TEST© will deal for us by opening the browser we selected in the XML Test Suite Configuration file, this step is already covered.
- Step 1 is “Navigate to the Google home page”. In an ideal world, that would be enough. But remember that we must tell DASH TEST©, that it must wait for the page to load completely before going to the next step. That means that this step is really 2 steps, navigate and then verify that the component we use next has already been loaded. Therefore, we will do the following:

On the Step Number (SN) column write “1a”, the first part of the first step. Then on the Step Description (SD) column, write “Navigate to Google home page”.

	A	B	C	D
1	EX	SN	SD	AP
2	N	0	Test the Google search page	
3	Y	1a	Navigate to Google home page	
4	Y			
5	Y			

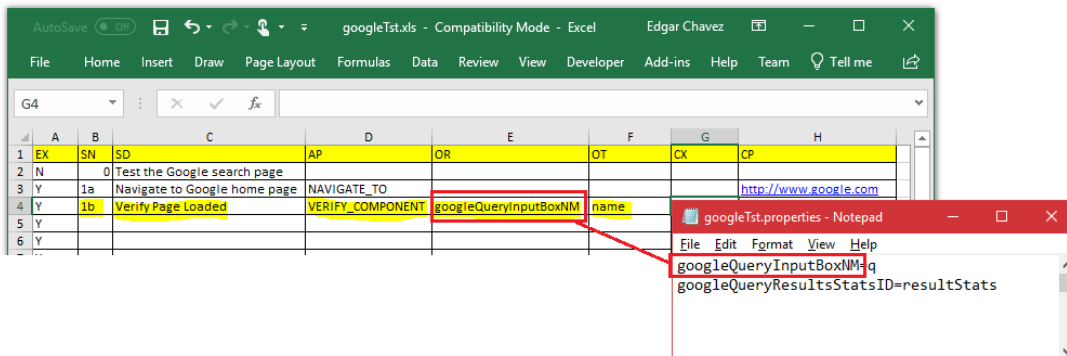
On the Action to Perform (AP) column, we must give the name of the DASH TEST© Component that will perform the action of “Navigate To” the Google home page. For more information, check [The DASH TEST© Component Library](#) later in this manual.

The component that we will use is actually “NAVIGATE_TO”. Navigate to only requires a parameter, that is the URL you want the browser to get. At the Component Parameter (CP) column, enter the google URL.

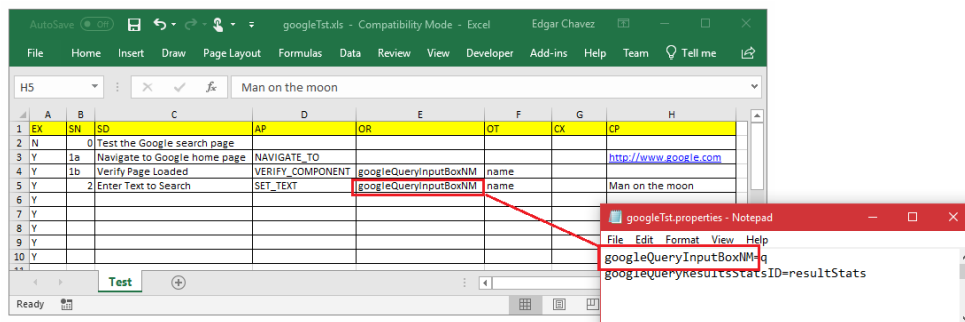
	A	B	C	D	E	F	G	H
1	EX	SN	SD	AP	OR	OT	CX	CP
2	N	0	Test the Google search page					
3	Y	1a	Navigate to Google home page	NAVIGATE_TO				http://www.google.com
4	Y							
5	Y							
6	Y							

- The next step is 1b, here we will deal with the verification that the page has been loaded completely before continuing execution.

The DASH TEST© Component we will use is “VERIFY_COMPONENT”. This component requires the Object Reference in the Page Object Reference file to the element we will wait for, the Search bar. Also, the Object Reference Type used to identify the element in the Web Page.



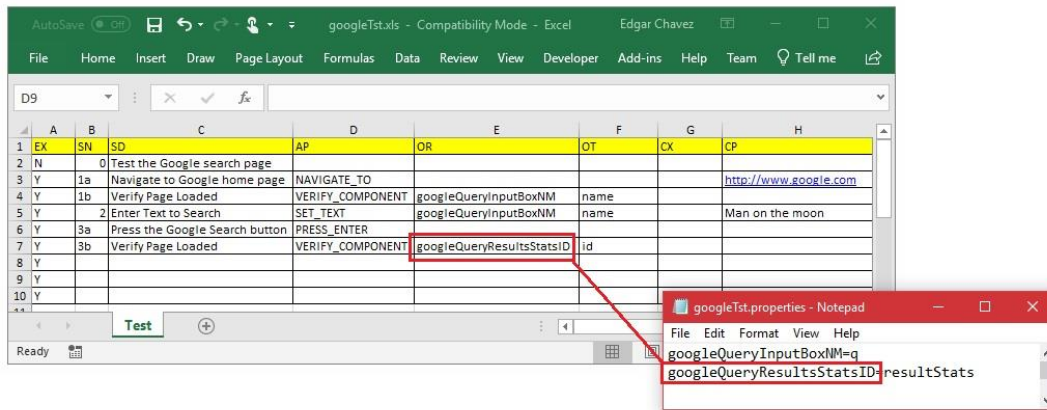
- Now to Step 2. Once the page is loaded, we must enter the search term in the Search bar. The component used is “SET_TEXT”. We must provide the Object Reference (OR) column, in the Page Object Reference file to the element we will write into, the Search bar. Also, the Object Reference Type (OT) column, used to identify the element in the Web Page and the text “Man on the moon” at the CP column.



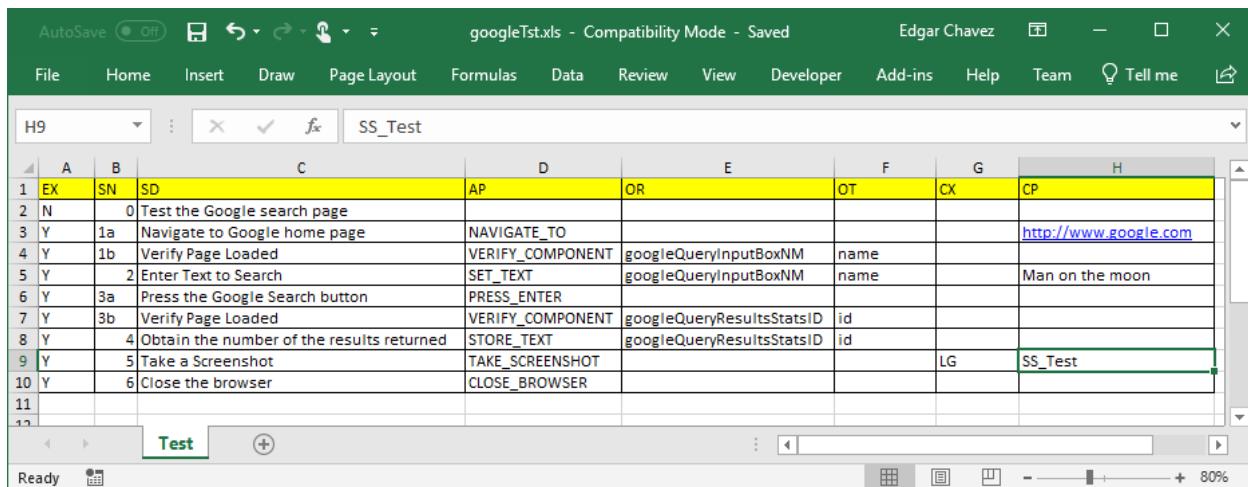
- Step 3 will require also of two parts “Press the Enter key” and then wait until the search is completed. To know the search is completed, we can use the search results total number as the Expected Condition.

Step 3a is Press the Enter key. Using the component named “PRESS_ENTER”.

Step 3b will verify that the object we referenced as “googleQueryResultsStatsID” has already been loaded.



- Step 4 is “Obtain the number of results that matched term”, we can use the “STORE_TEXT” component. It will save the value into a variable in memory and display it on the exit report. DASH TEST© needs the Object Reference (OR) in the Page Object Reference file where the value will be taken from, “googleQueryResultsStatsID”. Also, the Object Reference Type (OT) used to identify the element in the Web Page, the id.
- Step 5 is TAKE_SCREENSHOT. This component works in two ways. If you leave the Auxiliary Component Parameter (CX) column empty, DASH TEST© will take a screenshot of only the visible area of the Current Page. However, if you add the LG command in the CX column, DASH TEST© will screenshot the FULL PAGE, by scrolling down all the way to the bottom. On the CP column, enter the name of the screenshot. If you do not name the screenshot, DASH TEST will name it automatically using a timestamp with the prefix “DTSS_”.
- The last Step 6 is close the browser. Use the “CLOSE_BROWSER” component.



Your script is ready. Follow the steps to create an [XML Test Suite Configuration](#) file and [execute an Automated Test Script](#). Also, you can check the result of the execution at the [DASH TEST© reports](#).

The DASH TEST[®] Component Library.

Every Web Page is built with elements used to display images, texts, hyperlinks, tables, lists, buttons, text boxes, etc. DASH TEST[®] uses a library of Components to interact with these elements.

This is the current DASH TEST[®] Components Library index.

<u>The DASH TEST[®] Component Library.</u>	40
<u>Action Components.</u>	43
<u>ACTION KEYS</u>	43
<u>CLEAR TEXT</u>	43
<u>CLICK ALERT IF PRESENT</u>	43
<u>CLICK COMPONENT</u>	44
<u>CLICK COMPONENT BY PARTIAL TEXT</u>	44
<u>CLICK COMPONENT BY TEXT</u>	45
<u>CLICK COMPONENT IF PRESENT</u>	45
<u>CLICK COMPONENT IF STATUS</u>	46
<u>CLICK COMPONENT IF TEXT</u>	46
<u>CLICK COMPONENT TO STATUS</u>	47
<u>CLOSE BROWSER</u>	47
<u>CLOSE CHILD TAB</u>	48
<u>COUNT ELEMENTS IN LIST</u>	48
<u>COUNT OPEN TABS</u>	49
<u>FOCUS ON COMPONENT</u>	49
<u>HOVER COMPONENT</u>	50
<u>HOVER COMPONENT BY TEXT</u>	50
<u>LOCATE IN COMBO</u>	51
<u>LOG ENTRY</u>	51
<u>LOG STORED TEXT</u>	51
<u>LOG STORED NUMBER</u>	52
<u>NAVIGATE TO</u>	52
<u>PERFORM OPERATION</u>	53
<u>PRESS KEYS</u>	53
<u>RETURN COMPONENT HTML CLASS NAME</u>	54
<u>RETURN COMPONENT HTML PROPERTY</u>	54
<u>SELECT COMBO</u>	55
<u>SELECT COMBO BY INDEX</u>	55
<u>SELECT COMBO BY TEXT</u>	56

<u>SELECT_COMBO_BY_VALUE</u>	56
<u>SELECT_COMPONENT</u>	57
<u>SET_NUMBER</u>	58
<u>SET_TEXT</u>	58
<u>SET_TEXT_ENCLOSED_BY_AUX</u>	59
<u>SET_TEXT_NO_CLEAR</u>	60
<u>SET_WINDOW_SIZE</u>	60
<u>STORE_INPUT/STORE_INPUT_01 / STORE_INPUT_02</u>	61
<u>STORE_PARAMETER_NUMBER/STORE_PARAMETER_NUMBER01/STORE_PARAMETER_NUMBER02</u>	62
<u>STORE_PARAMETER_TEXT</u>	62
<u>STORE_NUMBER</u>	63
<u>STORE_TEXT</u>	63
<u>STORE_TEXT_IF_COMPONENT_PRESENT</u>	64
<u>SWITCH_TO_CHILD_TAB</u>	64
<u>SWITCH_TO_PARENT_TAB</u>	65
<u>TAKE_SCREENSHOT</u>	65
<u>USE_STORED_TEXT</u>	66
<u>USE_STORED_NUMBER</u>	67
<u>Verification Components</u>	68
<u>VERIFY_COMPONENT</u>	68
<u>VERIFY_COMPONENT_BY_TEXT</u>	68
<u>VERIFY_COMPONENT_BY_PARTIAL_TEXT</u>	68
<u>VERIFY_COMPONENT_CONTAINS_TEXT</u>	69
<u>VERIFY_IF_COMPONENT_STATUS_IS</u>	69
<u>Wait Components</u>	71
<u>WAIT_REFRESH</u>	71
<u>WAIT_UNTIL_TEXT_IS_EQUAL_TO</u>	71
<u>WAIT_WHILE_TEXT_IS_EQUAL_TO</u>	72
<u>Assert Components</u>	73
<u>ASSERT_COMPONENT_IS_SELECTED</u>	73
<u>ASSERT_COMPONENT_IS_PRESENT</u>	73
<u>ASSERT_COMPONENT_TEXT_FEATURE</u>	74
<u>ASSERT_CONTAINS</u>	74
<u>ASSERT_CONTAINS_IF_COMPONENT_PRESENT</u>	75
<u>ASSERT_CONTAINS_IF_COMPONENT_PRESENT_AND_NOT_AUX_TEXT</u>	75

<u>ASSERT CONTAINS INPUT</u>	76
<u>ASSERT CONTAINS STORED TEXT</u>	77
<u>ASSERT CONTAINS STORED NUMBER</u>	77
<u>ASSERT EQUALS</u>	78
<u>ASSERT EQUALS IF COMPONENT PRESENT</u>	78
<u>ASSERT EQUALS IF COMPONENT PRESENT AND NOT AUX TEXT</u>	79
<u>ASSERT EQUALS INPUT</u>	80
<u>ASSERT EQUALS NULL</u>	80
<u>ASSERT EQUALS TO STORED TEXT</u>	81
<u>ASSERT EQUALS TO STORED NUMBER</u>	81
<u>ASSERT HTML ATTRIBUTE CONTAINS PARAMETER</u>	82
<u>ASSERT HTML ATTRIBUTE EQUALS PARAMETER</u>	83
<u>ASSERT NOT CONTAINS</u>	83
<u>ASSERT STARTS WITH</u>	84

Action Components.

ACTION KEYS

DASH TEST® will simulate as if the user pressed a key in the keyboard. These components take no parameters.

- Uses Object Reference: NO. Leave blank.
- Uses Object Type: NO. Leave blank.
- Uses Component Parameter: NO. Leave blank.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Press Enter key	PRESS_ENTER				
Y	2	Press Escape key	PRESS_ESC				
Y	3	Press Tab key	PRESS_TAB				

[Back to DASH TEST® Component Library index](#)

CLEAR_TEXT

Clears the contents of a text box.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: No.

Examples:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Clear User Name	CLEAR_TEXT	usernameNm	name		

Object File:

usernameNm=< A 'name' reference in the Web Page using inspector tool >

[Back to DASH TEST® Component Library index](#)

CLICK_ALERT_IF_PRESENT

Sometimes, performing an action on the AUT, create some critical conditions, that sometimes require the user attention, that triggers alert messages that must be either dismissed or accepted. This component tells DASH TEST® to perform a click action on an alert box, only if the alert message is

present in the AUT. If the alert is not present in the AUT, the step is just ignored without reporting an error.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: NO. Leave blank.
- Uses Component Parameter: Yes. You must tell if the alert is accepted or not.

The valid responses to the alert message are ACCEPT or DISMISS.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	If displayed, Accept the alert message.	CLICK_ALERT_IF_PRESENT				ACCEPT

Object File:

loginButtonNm=< A 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

CLICK_COMPONENT

This component tells DASH TEST® to perform a click action on an object in the AUT, like a button.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: NO. Leave blank.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Press Login button	CLICK_COMPONENT	loginButtonNm	name		

Object File:

loginButtonNm=< A 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

CLICK_COMPONENT_BY_PARTIAL_TEXT

This component tells DASH TEST® to look for the first “clickable” object that has PART of the text passed as parameter, and perform a Click action on it, like a link. If the object is disabled, DASH TEST® will ignore it.

- Uses Object Reference: NO. Leave blank.
- Uses Object Type: NO. Leave blank.
- Uses Component Parameter: YES. Partial reference to locate the object, that will do the action.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Press "Sign In" button	CLICK_COMPONENT_BY_PARTIAL_TEXT				Sign

[Back to DASH TEST® Component Library index](#)

CLICK_COMPONENT_BY_TEXT

This component tells DASH TEST® to look for the first "clickable" object that has the same text passed as parameter, and perform a Click action on it, like a link. If the object is disabled, DASH TEST® will ignore it.

- Uses Object Reference: NO. Leave blank.
- Uses Object Type: NO. Leave blank.
- Uses Component Parameter: YES. Type in the text shown by the "clickable" object.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Press Login button	CLICK_COMPONENT_BY_TEXT				Login

[Back to DASH TEST® Component Library index](#)

CLICK_COMPONENT_IF_PRESENT

This component tells DASH TEST® to perform a click action on an object, like a button, only if this object is present in the AUT. If the object is not present in the AUT, the step is just ignored without reporting an error.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: NO. Leave blank.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Press Login button	CLICK_COMPONENT_IF_PRESENT	loginButtonNm	name		

Object File:

loginButtonNm=< A 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

CLICK_COMPONENT_IF_STATUS

This component tells DASH TEST® to perform a click action on an object in the AUT, like a button only if the Status indicated on the selected HTML Attribute, matches the argument provided in the Component Parameter column.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: YES. The expected Status (true or false).
- Uses Component Auxiliary Parameter: YES. Must contain the name of the HTML Attribute that contains the status value.

Example:

```
role="button" aria-disabled="false">
```

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Press Login button	CLICK_COMPONENT_IF_STATUS	loginButtonNm	name	aria-disabled	true

Object File:

loginButtonNm=< A 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

CLICK_COMPONENT_IF_TEXT

This component tells DASH TEST® to perform a click action on an object in the AUT, like a button only if the Object contains the text as described on the Column Parameter (CP_01).

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: YES. The text as displayed by the object.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Press Login button	CLICK_COMPONENT_IF_TEXT	loginButtonNm	name		LOGIN

Object File:

loginButtonNm=< A 'name' reference in the Web Page, using inspector tool >

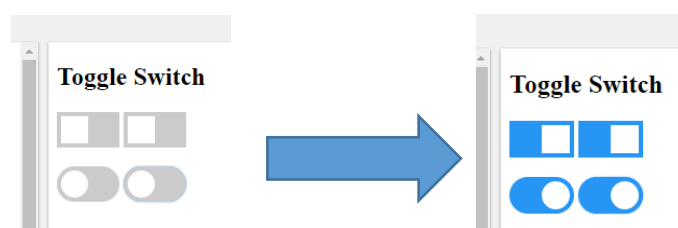
[Back to DASH TEST® Component Library index](#)

CLICK_COMPONENT_TO_STATUS

This component tells DASH TEST® to perform a click action on an object in the AUT, like a button, until its status matches the Boolean condition entered in the Component Parameter column.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: YES. Must be one of the following: TRUE or FALSE. Any other value would cause an Exception Error.

Example:



If the current toggle switch status of “OFF”, its Boolean value is FALSE. Thus, this component will change the status until it matches the Component Parameter value.

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Toggle Switch to the ON position.	CLICK_COMPONENT_TO_STATUS	powerButtonNm	name		TRUE

Object File:

powerButtonNm=< A 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

CLOSE_BROWSER

DASH TEST® will close the browser. Be careful to not placing the component in a script that will run in the middle of a Test Suite, since it will end the Web Driver session and terminate all the execution.

- Uses Object Reference: NO. Leave blank.
- Uses Object Type: NO. Leave blank.
- Uses Component Parameter: NO. Leave blank.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Close the browser	CLOSE_BROWSER				

[Back to DASH TEST® Component Library index](#)

CLOSE_CHILD_TAB

DASH TEST® will move focus into a CHILD tab in the current browser window and close the tab, without ending the test execution. If you want to end execution, you must use CLOSE_BROWSER.

NOTE: This component will NOT open new tabs. Use this component when there is only ONE Parent tab and ONE Child tab.

- Uses Object Reference: NO. Leave blank.
- Uses Object Type: NO. Leave blank.
- Uses Component Parameter: NO. Enter the tab name.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Switch focus to Parent tab	CLOSE_CHILD_TAB				

[Back to DASH TEST® Component Library index](#)

COUNT_ELEMENTS_IN_LIST

DASH TEST® will count the number of items that are visible and displayed in a table list. Obviously, if there are several pages in the table, the count will only throw the total found in the current page. Result is displayed in the log and in the report.

Optionally, by using the command **SAVE** in the CP column, you can store into a variable in memory the number of rows returned.

- Uses Object Reference: YES, a reference to an object in the Object Repository. Note that this is not the identification of the table, this is a common object identification that is repeated in every entry of the table.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: Yes. Optional.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Count items for sale	COUNT_ELEMENTS_IN_LIST	itemsForSaleNm	name		
Y	1	Count items in the storage, save count in memory.		itemsForSaleNm	name		SAVE

Object File:

itemsForSaleNm=< A 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

COUNT_OPEN_TABS

DASH TEST® will obtain the identifications of all currently open tabs and assign them a hierarchy PARENT/CHILD. Use this component AFTER a child tab has been open by the AUT.

NOTE: Use this component before switching between tabs in the browser.

- Uses Object Reference: NO. Leave blank.
- Uses Object Type: NO. Leave blank.
- Uses Component Parameter: NO. Leave blank.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Obtain the ID of all open tabs	COUNT_OPEN_TABS				

[Back to DASH TEST® Component Library index](#)

FOCUS_ON_COMPONENT

This component is used to make DASH TEST® set focus into an object in the AUT, without performing a Click action.

NOTE: The difference with SELECT_COMPONENT is that an index is not required.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: NO.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Set focus on the User Name Textbox	FOCUS_ON_COMPONENT	usernameNm	name		

[Back to DASH TEST® Component Library index](#)

HOVER_COMPONENT

DASH TEST® will perform a hover action on an object in the Web Page. This action activates any feature that is enabled when the mouse pointer crosses over the object, like a pop-up menu or a tool tip. But without selecting or setting the focus into it.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: NO. Leave blank.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Hover over menu option	HOVER_COMPONENT	homeMenuNm	name		

Object File:

homeMenuNm=< A 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

HOVER_COMPONENT_BY_TEXT

DASH TEST® will perform a hover action on an object in the Web Page, identified by the text shown on the AUT. This action activates any feature that is enabled when the mouse pointer crosses over the object, like a pop-up menu or a tool tip. But without selecting or setting the focus into it.

- Uses Object Reference: NO. Leave blank.
- Uses Object Type: NO. Leave blank.
- Uses Component Parameter: YES. The text displayed by the element on the AUT.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Hover over menu option	HOVER_COMPONENT_BY_TEXT				Home

Object File:

homeMenuNm=< A 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

LOCATE_IN_COMBO

This component tells DASH TEST® to locate an expected exact test in a combo box element and select it. Use this component as an alternative to SELECT_COMBO, when that component doesn't work as expected.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: YES. You need to tell DASH TEST® what option you are looking for and select.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Locate Option in Combo	LOCATE_IN_COMBO	timeZoneNm	name		Mountain Timezone

Object File:

timeZoneNm=< A 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

LOG_ENTRY

You can use this component to send a comment to the log.

- Uses Object Reference: NO. Leave blank.
- Uses Object Type: NO. Leave blank.
- Uses Component Parameter: YES. Your comment to the log.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Send stored text to log	LOG_ENTRY				Hello

[Back to DASH TEST® Component Library index](#)

LOG_STORED_TEXT

You can use this component to allow DASH TEST® the value stored by STORE_TEXT to a log.

- Uses Object Reference: NO. Leave blank.
- Uses Object Type: NO. Leave blank.
- Uses Component Parameter: NO. Leave blank.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Send stored text to log	LOG_STORED_TEXT				

Object File:

customerFullNameNm=< A 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

LOG_STORED_NUMBER

You can use this component to allow DASH TEST® the value stored by STORE_NUMBER to a log.

- Uses Object Reference: NO. Leave blank.
- Uses Object Type: NO. Leave blank.
- Uses Component Parameter: NO. Leave blank.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Send stored value to log	LOG_STORED_NUMBER				

Object File:

unitPriceNameNm=< A 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

NAVIGATE_TO

Use this component to tell DASH in what URL the AUT is located.

- Uses Object Reference: NO. Leave blank.
- Uses Object Type: NO. Leave blank.
- Uses Component Parameter: YES, the http URL where the AUT is located.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Navigate to URL	NAVIGATE_TO				http://www.google.com

[Back to DASH TEST® Component Library index](#)

PERFORM_OPERATION

DASH TEST® use the value stored by using components STORE_INPUT_01/ STORE_INPUT_02 or STORE_INPUT_PARAMETER_01/ STORE_INPUT_PARAMETER_02 to perform the mathematical operation indicated in the Component Parameter column. The result will display in the log and in the report.

NOTE: You must keep attention to the numeric variable, where the values are stored, when using them to perform a mathematical operation.

- Uses Object Reference: NO.
- Uses Object Type: NO.
- Uses Component Parameter: YES. You must specify the operation you want to perform.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Sum stored data display result	PERFORM_OPERATION				ADDITION
Y	2	Subtract stored data display result	PERFORM_OPERATION				SUBTRACTION
Y	3	Multiply stored data display result	PERFORM_OPERATION				MULTIPLICATION
Y	4	Divide stored data display result	PERFORM_OPERATION				DIVISION

NOTE: Available operations are ADDITION, SUBTRACTION, MULTIPLICATION and DIVISION.

[Back to DASH TEST® Component Library index](#)

PRESS_KEYS

DASH TEST® will simulate as if the user pressed an alphanumeric key or several keys one by one, given on the CP_01 column, in the keyboard. The difference with SET_TEXT is that keys are pressed but they are not associated to any input object in the AUT. If you want to enter text into an input box, use SET_TEXT.

- Uses Object Reference: NO. Leave blank.
- Uses Object Type: NO. Leave blank.
- Uses Component Parameter: Yes. The key or keys DASH TEST® will simulate pressing.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Press the "x" key	PRESS_KEYS				X
Y	2	Press the keys "N" "A" "M" "E"	PRESS_KEYS				NAME

[Back to DASH TEST® Component Library index](#)

RETURN_COMPONENT_HTML_CLASS_NAME

DASH TEST® will obtain the attribute TEXT value from the object indicated in the Object Reference column. The attribute from where the value will be obtained is indicated in the Component Parameter column. This is different from the RETURN_COMPONENT_HTML_PROPERTY component, because it will only return the value associated with the *class* **TAG**.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: NO.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Press action button	RETURN_COMPONENT_HTML_CLASS_NAME	actionButtonNm	name		

Object File:

actionButtonNm=< A 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

RETURN_COMPONENT_HTML_PROPERTY

DASH TEST® will obtain the attribute TEXT value from the object indicated in the Object Reference column. The attribute from where the value will be obtained is indicated in the Component Parameter column. This is different from RETURN_COMPONENT_HTML_CLASS_NAME, because it can return the value associated with any **html TAG** associated with the object indicated at the Object Reference column. Information will be displayed in the log and in the report.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: YES. You must tell the HTML Attribute where data will be taken from.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Is the action button pressed?	RETURN_COMPONENT_PROPERTY_VALUE	actionButtonNm	name		aria-pressed

Object File:

actionButtonNm=< A 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

SELECT_COMBO

Use this component to tell DASH TEST® to select a value from a Combo List type of object.

LOCATE_IN_COMBO is an alternate solution, when this component doesn't work as expected.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: Yes. You need to tell DASH TEST® what the value is you want to select.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Select from Combo	SELECT_COMBO	timeZoneNm	name		Mountain Timezone

Object File:

timeZoneNm=< A 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

SELECT_COMBO_BY_INDEX

Use this component to tell DASH TEST® to select an element by index location, that matches the parameter in the Combo List.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: Yes, optional. You need to tell DASH TEST® what the index of the option is you want to select.

NOTE: If you leave the Component Parameter column empty, enter 0 or 1, DASH TEST® will select the first option in the Combo list. If the index contains a value that is not a number, DASH TEST® will throw an error.

Example:

```
<option value="10">ANGUILLA </option>
<option value="234">ANTARCTICA </option>
<option value="1">ANTIGUA AND BARBUDA </option>
```

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Select from Combo	SELECT_COMBO_BY_INDEX	locationNM	name		2

DASH TEST® will select the second element in the list, in the example is “ANTARTICA”.

Object File:

locationNM=< A ‘name’ reference in the Web Page to the Combo List, using inspector tool >

[Back to DASH TEST® Component Library index](#)

SELECT_COMBO_BY_TEXT

Use this component to tell DASH TEST® to select an element exactly by the text that matches the parameter in the Combo List.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: Yes. The text is you want to select.

Example:

```
<option value="10">ANGUILLA </option>
<option value="234">ANTARCTICA </option>
<option value="1">ANTIGUA AND BARBUDA </option>
```

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Select from Combo	SELECT_COMBO_BY_TEXT	locationNM	name		ANTARTICA

Object File:

locationNM=< A ‘name’ reference in the Web Page to the Combo List, using inspector tool >

[Back to DASH TEST® Component Library index](#)

SELECT_COMBO_BY_VALUE

Use this component to tell DASH TEST® to select an element by the “value” HTML Attribute of the option in the Combo List.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.

- Uses Component Parameter: Yes. You need to tell DASH TEST® what the “value” is you want to select.

Example:

```
<option value="10">ANGUILLA </option>
<option value="234">ANTARCTICA </option>
<option value="1">ANTIGUA AND BARBUDA </option>
```

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Select from Combo	SELECT_COMBO_BY_VALUE	locationNM	name		234

Object File:

locationNM=< A ‘name’ reference in the Web Page to the Combo List, using inspector tool >

[Back to DASH TEST® Component Library index](#)

SELECT_COMPONENT

This component is used to make DASH TEST® set focus into an object in the AUT, without performing a Click action.

NOTE: If the index contains a value that is not a number, DASH TEST® will throw an error.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: YES, optional. You can provide an index number for the object in the HTML structure, when there are several instances of it in the page. Leave it blank if there is only one.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Select User Name Textbox	SELECT_COMPONENT	usernameNm	name		

DASH TEST® will select the **FIRST matching** object on the UAT, the referenced object the OR_4 parameter.

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Select User Name Textbox	SELECT_COMPONENT	usernameNm	name		0

DASH TEST® will select the **FIRST matching** object on the UAT, the referenced object the OR_4 parameter.

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Select User Name Textbox	SELECT_COMPONENT	usernameNm	name		1

DASH TEST® will select the **FIRST matching** object on the UAT, the referenced object the OR_4 parameter.

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Select User Name Textbox	SELECT_COMPONENT	usernameNm	name		2

DASH TEST® will select the **SECOND matching** object on the UAT, the referenced object the OR_4 parameter.

Object File:

usernameNm=< A 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

SET_NUMBER

This component tells DASH TEST® to input numeric information into a numeric input box object. DASH TEST® will clear the contents of the input box, then enter the information provided as a parameter. The input information displays in the log.

NOTE: Use this component only when the input box can't handle values entered as Strings.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: Yes. You need to tell DASH TEST® what input you want to enter.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Enter number of items sold	SET_NUMBER	itemsSoldNm	name		2500

Object File:

itemsSoldNm=< A 'name' reference in the Web Page using inspector tool >

[Back to DASH TEST® Component Library index](#)

SET_TEXT

This component tells DASH TEST® to input information into a text box object. DASH TEST® will clear the contents of the text box, then enter the information provided as a parameter. The input information displays in the log.

NOTE 1: If the text you're sending is a password and want to avoid revealing it in the log, be sure to let know DASH TEST® what it is, by including the word "password" in the text entered in the Step Description (SD_2) column.

NOTE 2: All values entered in the cells MUST be entered as text or as number. DASH TEST® will convert formats such as dates currency or percentage to double numbers. DASH TEST® doesn't interpret formulas and will return an error.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: Yes. You need to tell DASH TEST® what input you want to enter.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Enter User Name	SET_TEXT	usernameNm	name		JohnSmith

Object File:

usernameNm=< A 'name' reference in the Web Page using inspector tool >

[Back to DASH TEST® Component Library index](#)

SET_TEXT_ENCLOSED_BY_AUX

This component tells DASH TEST® to input information into a text box object by enclosing it by the text input in the Component Auxiliary column (CX_6). DASH TEST® will clear the contents of the text box, then enter the information provided as a parameter. The input information displays in the log.

This is useful when, in some search boxes, enclosing the text in a special character, narrows the searching criteria. Check with the developer of the AUT if this functionality is available.

NOTE 1: All values entered in the cells MUST be entered as text or as number. DASH TEST® will convert formats such as dates currency or percentage to double numbers. DASH TEST® doesn't interpret formulas and will return an error.

NOTE2: The results of the query will depend on the efficiency of the search engine.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: Yes. You need to tell DASH TEST® what input you want to enter.
- Uses Component Auxiliary Parameter: Yes. You must enter the enclosing text.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
------	------	------	------	------	------	------	-------

Y	1	Enter Search Term	SET_TEXT_ENCLOSED_BY AUX	searchBoxNm	name	"	Maximum Speed
---	---	-------------------	--------------------------	-------------	------	---	---------------

Object File:

searchBoxNm=< A 'name' reference in the Web Page using inspector tool >

[Back to DASH TEST® Component Library index](#)

SET_TEXT_NO_CLEAR

Same as the previous component, but this one will not clear the contents of the text box before entering the information provided as a parameter. The input information displays in the log.

NOTE 1: If the text you're sending is a password and want to avoid revealing it in the log, be sure to let know DASH TEST® what it is, by including the word "password" in the text entered in the Step Description (SD_2) column.

NOTE 2: All values entered in the cells MUST be entered as text or as number. DASH TEST® will convert formats such as dates currency or percentage to double numbers. DASH TEST® doesn't interpret formulas and will return an error.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: Yes. You need to tell DASH TEST® what input you want to enter.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Enter User Name	SET_TEXT_NO_CLEAR	usernameNm	name		JohnSmith

Object File:

usernameNm=< A 'name' reference in the Web Page using inspector tool >

[Back to DASH TEST® Component Library index](#)

SET_WINDOW_SIZE

Use this component to change the browser window size where browser is running.

- Uses Object Reference: NO. Leave blank.
- Uses Object Type: NO. Leave blank.
- Uses Component Parameter: YES. The size in whole numeric PIXELS in the form WIDTH: HEIGHT. If you provide the option "MAX", DASH TEST® will change the dimensions of the window to the maximum specified in the current system configuration, for the display where DASH TEST® is executing.

NOTE: Be sure to type your window dimensions using the correct notation #pixelsWIDE:#pixelsTALL, like in the example.

Example:

Script File:

Resize window to the specified dimensions

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Change window size	SET_WINDOW_SIZE				800:600

Maximize Window

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Change window size	SET_WINDOW_SIZE				MAX

[Back to DASH TEST® Component Library index](#)

STORE_INPUT/STORE_INPUT_01 / STORE_INPUT_02

For the STORE_INPUT component, DASH TEST® will read a value from an Input element type in the AUT, from the object indicated in the Object Reference column. The value is stored into memory and can be used with the USE_STORED_TEXT or USE_STORED_NUMBER components. If there is already a value in the variable, it will be replaced.

For STORE_INPUT_01 and STORE_INPUT_02 components, the values are stored into memory as numeric variables that can be used to perform mathematical operations, with the PERFORM_OPERATION component.

NOTE: You must keep attention to the numeric variable, where the values are stored, when using them to perform a mathematical operation.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: YES. The name of the html Attribute that contains the value to store.

Example:

a. STORE_INPUT

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Save File Name	STORE_INPUT	fileNameInputBoxNm	name		value

Object File:

fileNameInputBoxNm=< A 'name' reference in the Web Page, using inspector tool >

b. STORE_INPUT_01 and STORE_INPUT_02

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Save File Name	STORE_INPUT_01	itemsSold01Nm	name		value
Y	1	Save File Name	STORE_INPUT_02	itemsSold02Nm	name		value

Object File:

itemsSold01Nm=< A 'name' reference in the Web Page, using inspector tool >

itemsSold02Nm=< A 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

STORE_PARAMETER_NUMBER/STORE_PARAMETER_NUMBER01/STORE_PARAMETER_NUMBER02

For STORE_PARAMETER, DASH TEST® will save a numeric value, directly taken from the Component Parameter column into a temporary variable. You can later use the value stored to perform validations against the AUT.

STORE_PARAMETER_NUMBER01 and STORE_PARAMETER_NUMBER02, you can later use the value stored to perform validations against the AUT or to perform mathematical operations.

- Uses Object Reference: NO. Leave blank.
- Uses Object Type: NO. Leave blank.
- Uses Component Parameter: YES. You must provide the value to store, must be a NUMBER.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Store into NUMBER from CP_01	STORE_PARAMETER_NUMBER				250
Y	2	Store into NUMBER01 from CP_01	STORE_PARAMETER_NUMBER01				250
Y	3	Store into NUMBER02 from CP_01	STORE_PARAMETER_NUMBER02				10

[Back to DASH TEST® Component Library index](#)

STORE_PARAMETER_TEXT

DASH TEST® will save a text value, directly taken from the Component Parameter column into a temporary variable. You can later use the value stored to perform validations against the AUT.

- Uses Object Reference: NO. Leave blank.

- Uses Object Type: NO. Leave blank.
- Uses Component Parameter: YES. You must provide the value to store, must be a TEXT.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Store the TEXT input parameter	STORE_INPUT_PARAMETER_TEXT				Organism

[Back to DASH TEST® Component Library index](#)

STORE_NUMBER

DASH TEST® will read a numeric value from the AUT, in the object indicated in the Object Reference column and display it in the log. The value is stored into memory as a String variable and can be used with the USE_STORED_NUMBER component. If there is already a value in the variable, it will be replaced. STORE_NUMBER doesn't work with input textboxes.

If the value you want to store is a date, use STORE_TEXT.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: NO. Leave blank.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Save Price Value	STORE_NUMBER	unitPriceNm	name		

Object File:

unitPriceNm=< A 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

STORE_TEXT

DASH TEST® will read a non-numeric value from the AUT, in the object indicated in the Component Name (OR_4) column. The value is stored into memory as a String variable and can be used with the USE_STORED_TEXT component. If there is already a value in the variable, it will be replaced. STORE_TEXT doesn't work with input textboxes, use STORE_INPUT instead.

If the value you want to store is a date, use STORE_TEXT.

- Uses Object Reference: YES, a reference to an object in the Object Repository.

- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: NO. Leave blank.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Save Customer Name	STORE_TEXT	customerFullNameNm	name		

Object File:

customerFullNameNm=< A 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

STORE_TEXT_IF_COMPONENT_PRESENT

DASH TEST® will read a non-numeric value from the AUT, if the object indicated in the Component Name (OR_4) column is present in the AUT. The value is stored into memory as a String variable and can be used with the USE_STORED_TEXT component. If there is already a value in the variable, it will be replaced. STORE_TEXT doesn't work with input textboxes, use STORE_INPUT instead. If the Component is not present, the operation is ignored, without reporting the issue.

You can store a DATE using this component.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: NO. Leave blank.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Save Customer Name	STORE_TEXT_IF COMPONENT_PRESENT	customerFullNameNm	name		

Object File:

customerFullNameNm=< A 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

SWITCH_TO_CHILD_TAB

DASH TEST® will move focus into a new CHILD tab in the current browser window.

NOTE: This component will NOT open new tabs. Use this component when there is only ONE Parent tab and ONE Child tab.

- Uses Object Reference: NO. Leave blank.
- Uses Object Type: NO. Leave blank.
- Uses Component Parameter: NO. Enter the tab name.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Switch focus to Child tab	SWITCH_TO_CHILD_TAB				

[Back to DASH TEST® Component Library index](#)

SWITCH_TO_PARENT_TAB

DASH TEST® will move focus into the PARENT tab in the current browser window. The parent tab is the original web browser application. Be careful, not to close this tab, because it will drop all execution.

NOTE: This component will NOT open new tabs. Use this component when there is only ONE Parent tab and ONE Child tab.

- Uses Object Reference: NO. Leave blank.
- Uses Object Type: NO. Leave blank.
- Uses Component Parameter: NO. Enter the tab name.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Switch focus to Parent tab	SWITCH_TO_PARENT_TAB				

[Back to DASH TEST® Component Library index](#)

TAKE_SCREENSHOT

DASH TEST® will create a “jpg” file image of the currently displayed web page and save it in the following path: test-output\sshots\< testCaseID >.

This component works in two ways:

- By using the regular command, will take a screenshot of the currently visible area on the main display.
- By adding the LG option in the Auxiliary Component Parameter column, DASH TEST® will scroll down the full page, while taking screenshots one visible area at the time.

Note: You don’t need to supply the “jpg” file extension; DASH will attach it to the file for you.

- Uses Object Reference: YES, a reference to an object in the Object Repository.

- Uses Object Type: NO. Leave blank.
- Uses Component Parameter: Yes. The file name, without extension. If no name is provided, DASH TEST® will automatically name the file with the suffix “DTSS_” and a timestamp.
- Uses Parameter Auxiliary: Optionally YES. If you provide the option “LG”, DASH TEST® will take a full screenshot of the current page, if the page is longer than the current space visible on screen, there will be a scrolling down all the way to the bottom. DASH TEST® will create a composite image of the complete page.

Example:

Script File:

Normal screenshot

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Take Screenshot	TAKE_SCREENSHOT				screenshot_01

Long Screenshot

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Take Screenshot	TAKE_SCREENSHOT			LG	screenshot_01

[Back to DASH TEST® Component Library index](#)

USE_STORED_TEXT

DASH TEST® use the value stored by STORE_TEXT and use it as input, into a textbox object, indicated in the Object Reference column.

- Uses Object Reference: YES, a reference to an object in the Object Repository to the input box where you want to place the value stored in memory.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: NO. Leave blank.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Input Saved Customer Name	USE_STORED_TEXT	customerFullNameNm	name		

Object File:

customerFullNameNm=< A ‘name’ reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

USE_STORED_NUMBER

DASH TEST® use the value stored by STORE_NUMBER and use it as input, into a textbox object, indicated in the Object Reference column.

- Uses Object Reference: YES, a reference to an object in the Object Repository to the input box where you want to place the value stored in memory.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: NO. Leave blank.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Input Saved Item Price	USE_STORED_NUMBER	unitPriceNm	name		

Object File:

unitPriceNameNm=< A 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

Verification Components.

VERIFY_COMPONENT

DASH TEST® will confirm the presence of an object in the current Web Page. You can use this component to verify a web page has been completely loaded, to continue execution. DASH will wait for one minute before continuing execution.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: NO. Leave blank.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Is Unit Price Present?	VERIFY_COMPONENT	unitPriceNm	name		

Object File:

unitPriceNameNm=< A 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

VERIFY_COMPONENT_BY_TEXT

DASH TEST® will confirm the presence of an object in the current Web Page, by using the Text shown in the AUT. You can use this component to verify a web page has been completely loaded, to continue execution. DASH will wait for one minute before continuing execution.

- Uses Object Reference: NO. Leave blank.
- Uses Object Type: NO. Leave blank.
- Uses Component Parameter: YES. The text displayed by the element on the AUT.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Is Login Button Present?	VERIFY_COMPONENT_BY_TEXT				Login

[Back to DASH TEST® Component Library index](#)

VERIFY_COMPONENT_BY_PARTIAL_TEXT

DASH TEST® will confirm the presence of an object in the current Web Page, by using the Text shown in the AUT. You can use this component to verify a web page has been completely loaded, to continue execution. DASH will wait for one minute before continuing execution.

- Uses Object Reference: NO. Leave blank.
- Uses Object Type: NO. Leave blank.
- Uses Component Parameter: YES. The text displayed by the element on the AUT.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Is "Sign In" Button Present?	VERIFY_COMPONENT_BY_PARTIAL_TEXT				Sign

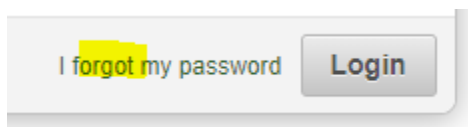
[Back to DASH TEST® Component Library index](#)

VERIFY_COMPONENT_CONTAINS_TEXT

DASH TEST® will confirm the presence of a specific text on an object in the current Web Page. You can use this component to verify a web page has been completely loaded, to continue execution. DASH will wait for one minute, for the text to display on the object, before continuing execution.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: YES. You must provide the exact text reference that eventually must display on the object.

Example:



Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Label contains the word "Login"?	VERIFY_COMPONENT_CONTAINS_TEXT	PWDHelpLabelNM	name		forgot

Object File:

PWDHelpLabelNM =< A 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

VERIFY_IF_COMPONENT_STATUS_IS

DASH TEST® verify and waits until the status, of the object referenced in the Component Name column, matches the status in the Component Parameter column. If after the wait, the component doesn't match the expected status, DASH TEST® will report it in the log and continue execution. No exception is thrown.

Accepted Component Parameter status values are: ENABLED or TRUE and DISABLED or FALSE.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: NO.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Is the button1 enabled?	VERIFY_IF_COMPONENT_STATUS_IS	Action1ButtonNm	name		ENABLED
Y	2	Is the button2 disabled?	VERIFY_IF_COMPONENT_STATUS_IS	Action2ButtonNm	name		FALSE

Object File:

actionButtonNm=< A 'name' reference in the Web Page using inspector tool >

[Back to DASH TEST® Component Library index](#)

Wait Components.

WAIT_REFRESH

DASH TEST® will make 10 second pause in execution. This component takes no parameters.

- Uses Object Reference: NO. Leave blank.
- Uses Object Type: NO. Leave blank.
- Uses Component Parameter: Optional. If you leave blank, DASH TEST® will wait for 10 seconds before continue execution or you can enter a value in milliseconds. NOTE: The numeric value must be given in TEXT format (String), to do so, in Excel enter a tilde ' before the number: '30000.

Example:

Script File:

DASH TEST® will for 10 seconds (default)

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Make a pause	WAIT_REFRESH				

DASH TEST® will for 30 seconds

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Make a pause	WAIT_REFRESH_CUSTOM				'30000

[Back to DASH TEST® Component Library index](#)

WAIT_UNTIL_TEXT_IS_EQUAL_TO

DASH TEST® will check for the current text displayed by the component and wait for one minute or until it changes to the text indicated by the Component Parameter column. If the value doesn't display after one minute, DASH will report the issue and continue execution.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: YES. The expected message.
- Uses Component Auxiliary Parameter: YES. Optionally, you can set the maximum wait time, in seconds, for the component to wait for the expected changes. The default wait time is 60 seconds.

Example:

In the first example, DASH TEST® will check for until the expected text displays, for until a minute has passed. After that, execution will continue, whether expected conditions were met or not.

In the second example, DASH TEST® will check for until the expected text displays, for until 2 minutes had passed. After that, execution will continue, whether expected conditions were met or not.

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Changes applying?	WAIT_UNTIL_TEXT_IS_EQUAL_TO	statusMsgNm	name		Applying changes
Y	2	Changes applying?	WAIT_UNTIL_TEXT_IS_EQUAL_TO	statusMsgNm	name	120	Applying changes

Object File:

statusMsgNm=< A 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

WAIT_WHILE_TEXT_IS_EQUAL_TO

DASH TEST® will check for the current text displayed by the component and wait for one minute or while it is equal to the text indicated by the Component Parameter column. If the value doesn't change after one minute, DASH will report the issue and continue execution.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: YES. The expected message.
- Uses Component Auxiliary Parameter: YES. Optionally, you can set the maximum wait time, in seconds, for the component to wait for the expected changes. The default wait time is 60 seconds.

Example:

In the first example, DASH TEST® will check while the expected text still displays, just for a minute. But if the text changed before, execution will continue.

In the second example, DASH TEST® will check while the expected text still displays, just for 2 minutes. But if the text changed before, execution will continue.

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Where changes applied?	WAIT_WHILE_TEXT_IS_EQUAL_TO	statusMsgNm	name		Changes Applied
Y	2	Where changes applied?	WAIT_WHILE_TEXT_IS_EQUAL_TO	statusMsgNm	name	120	Changes Applied

Object File:

statusMsgNm=< A 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

Assert Components.

ASSERT_COMPONENT_IS_SELECTED

DASH TEST® check to see if the object in the Component Name column is selected, like a radio button or a checkbox. If the object is not selected, an exception will be thrown.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: NO.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Is the radio selected?	ASSERT_COMPONENT_IS_SELECTED	radioButtonNm	name		

Object File:

radioButtonNm=< A 'name' reference in the Web Page using inspector tool >

[Back to DASH TEST® Component Library index](#)

ASSERT_COMPONENT_IS_PRESENT

DASH TEST® will validate that a component is present in the AUT. Pay attention to punctuation and letter case, DASH TEST® is case sensitive. DASH will wait for 5 seconds for value to match, before rising an exception, terminate the script and report the issue.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: NO. Leave blank.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Is Unit Price Visible?	ASSERT_COMPONENT_IS_PRESENT	unitPriceNm	name		

Object File:

unitPriceNameNm=< A 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

ASSERT_COMPONENT_TEXT_FEATURE

DASH TEST® will validate that a component's text attributes matches the value given in the CP_01 column. DASH will wait for 5 seconds for value to match, before rising an exception, terminate the script and report the issue.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: YES. The text attribute value.
- Uses Auxiliary Component Parameter: YES. You must tell what feature you want to assert.

To obtain the text attribute value, use the browser inspector and locate the following entry in the CSS properties:



Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Validate text color	ASSERT_COMPONENT_TEXT_FEATURE	govSysTextNm	name	COLOR	#aa0909
Y	1	Validate text color	ASSERT_COMPONENT_TEXT_FEATURE	govSysTextNm	name	SIZE	0.8
Y	1	Validate text color	ASSERT_COMPONENT_TEXT_FEATURE	govSysTextNm	name	WEIGHT	Bolder
Y	1	Validate text color	ASSERT_COMPONENT_TEXT_FEATURE	govSysTextNm	name	ALIGN	Center

Object File:

govSysTextNm=< A 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

ASSERT_CONTAINS

DASH TEST® will use this component to perform the validation where, the text object in the AUT contains part of the text passed in the CP_01 column. Use this component to compare values that are not in input text boxes. Pay attention to punctuation and letter case, DASH TEST® is case sensitive. DASH will wait for 5 seconds for value to match, before rising an exception, terminate the script and report the issue.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.

- Uses Component Parameter: YES. You need to tell DASH TEST® what the value you want compare is.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Validate Citizen Kane last words	ASSERT_CONTAINS	commentsNm	name		Rosebud

Object File:

commentsNm=< A 'name' reference in the Web Page using inspector tool >

[Back to DASH TEST® Component Library index](#)

ASSERT_CONTAINS_IF_COMPONENT_PRESENT

DASH TEST® will use this component to perform the validation where, the text object in the AUT has the exact same value passed in the CP_01 column, only if the text object is present. Use this component to compare values that are not in input text boxes. To make assertions on Input Textboxes, use ASSERT_EQUALS_INPUT or ASSERT_CONTAINS_INPUT. Pay attention to punctuation and letter case, DASH TEST® is case sensitive. DASH will wait for 5 seconds for value to match, before rising an exception, terminate the script and report the issue. If the object is not present in the AUT, the operation will be ignored, no issue will be reported.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: YES. What the value you want compare is.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Validate User ID	ASSERT_CONTAINS_IF_COMPONENT_PRESENT	usernameNm	name		JohnSmith

Object File:

usernameNm=< A 'name' reference in the Web Page using inspector tool >

[Back to DASH TEST® Component Library index](#)

ASSERT_CONTAINS_IF_COMPONENT_PRESENT_AND_NOT_AUX_TEXT

DASH TEST® will use this component to perform the validation where, the text object in the AUT has the exact same value passed in the CP_01 column, only if the text object is present and the text is not equal to the text in the Component Auxiliary (CX_6) column.

Use this component to compare values that are not in input text boxes. To make assertions on Input Textboxes, use ASSERT_EQUALS_INPUT or ASSERT_CONTAINS_INPUT. Pay attention to punctuation and letter case, DASH TEST® is case sensitive. DASH will wait for 5 seconds for value to match, before rising an exception, terminate the script and report the issue. If the object is not present in the AUT, the operation will be ignored, no issue will be reported.

This Component is useful, when is possible that the component has a value different that the expected, but not necessarily this is an error. For example, when performing a search, there are not element found and the system reports such status, but it is not a problem with the AUT. If the object displays a value equal to the text in the Auxiliary column, the operation is ignored, without reporting the issue.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: YES. You need to tell DASH TEST® what the value you want compare is.
- Uses Component Auxiliary Parameter: YES. You must enter the message that will be ignored by the assertion.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Validate User ID	ASSERT_CONTAINS_IF_COMPONENT_PRESENT_AND_NOT_AUX_TEXT	userNm	name	Not Found	JohnSmith

Object File:

userNm=< A 'name' reference in the Web Page using inspector tool >

[Back to DASH TEST® Component Library index](#)

ASSERT_CONTAINS_INPUT

DASH TEST® will use this component to perform a validation where an input text box object in the AUT contains part of the text value passed in the CP_01 column. Pay attention to punctuation and letter case, DASH TEST® is case sensitive. DASH will wait for 5 seconds for value to match, before rising an exception, terminate the script and report the issue.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: YES. You need to tell DASH TEST® what the value you want compare is.
- Uses Parameter Auxiliary: YES. You must provide the HTML Attribute where the value is stored in the Input Box.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Validate File Name used	ASSERT_CONTAINS_INPUT	fileNameInputBoxNm	name	value	Book.txt

Object File:

fileNameInputBoxNm=< A 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

ASSERT_CONTAINS_STORED_TEXT

DASH TEST® will use this component to perform a validation using the value stored in an internal variable, using the STORE_TEXT component, against an element in the AUT. Text contained in the variable can PARTIALLY MATCH the text in the AUT element. DASH will wait for 5 seconds for value to match, before rising an exception, terminate the script and report the issue.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: NO. Leave blank.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Validate variable text is contained AUT	ASSERT_CONTAINS_STORED_TEXT	welcomeTitleNm	name		

Object File:

welcomeTitleNm=< An 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

ASSERT_CONTAINS_STORED_NUMBER

DASH TEST® will use this component to perform a validation using the value stored in an internal variable, using the STORE_NUMBER component, against an element in the AUT. A numeric value contained in the variable can PARTIALLY MATCH a numeric value in the AUT element. DASH will wait for 5 seconds for value to match, before rising an exception, terminate the script and report the issue.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: NO. Leave blank.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Validate variable value is contained AUT	ASSERT_CONTAINS_STORED_NUMBER	welcomeTitleNm	name		

Object File:

welcomeTitleNm=< An 'name' reference in the Web Page, using inspector tool >

[Back to DASH TEST® Component Library index](#)

ASSERT_EQUALS

DASH TEST® will use this component to perform the validation where, the text object in the AUT has the exact same value passed in the CP_01 column. Use this component to compare values that are not in input text boxes. To make assertions on Input Textboxes, use ASSERT_EQUALS_INPUT or ASSERT_CONTAINS_INPUT. Pay attention to punctuation and letter case, DASH TEST® is case sensitive. DASH will wait for 5 seconds for value to match, before rising an exception, terminate the script and report the issue.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: YES. You need to tell DASH TEST® what the value you want compare is.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Validate User ID	ASSERT_EQUALS	usernameNm	name		JohnSmith

Object File:

usernameNm=< A 'name' reference in the Web Page using inspector tool >

[Back to DASH TEST® Component Library index](#)

ASSERT_EQUALS_IF_COMPONENT_PRESENT

DASH TEST® will use this component to perform the validation where, the text object in the AUT has the exact same value passed in the CP_01 column, only if the text object is present. Use this component to compare values that are not in input text boxes. To make assertions on Input Textboxes, use ASSERT_EQUALS_INPUT or ASSERT_CONTAINS_INPUT. Pay attention to punctuation and letter case, DASH TEST® is case sensitive. DASH will wait for 5 seconds for value to match, before rising an exception, terminate the script and report the issue. If the object is not present in the AUT, the operation will be ignored, no issue will be reported.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: YES. You need to tell DASH TEST® what the value you want compare is.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Validate User ID	ASSERT_EQUALS_IF_COMPONENT_PRESENT	usernameNm	name		JohnSmith

Object File:

usernameNm=< A 'name' reference in the Web Page using inspector tool >

[Back to DASH TEST® Component Library index](#)

ASSERT_EQUALS_IF_COMPONENT_PRESENT_AND_NOT_AUX_TEXT

DASH TEST® will use this component to perform the validation where, the text object in the AUT has the exact same value passed in the CP_01 column, only if the text object is present and the text is not equal to the text in the Component Auxiliary (CX_6) column.

Use this component to compare values that are not in input text boxes. To make assertions on Input Textboxes, use ASSERT_EQUALS_INPUT or ASSERT_CONTAINS_INPUT. Pay attention to punctuation and letter case, DASH TEST® is case sensitive. DASH will wait for 5 seconds for value to match, before rising an exception, terminate the script and report the issue. If the object is not present in the AUT, the operation will be ignored, no issue will be reported.

This Component is useful, when is possible that the component has a value different that the expected, but not necessarily this is an error. For example, when performing a search, there are not element found and the system reports such status, but it is not a problem with the AUT. If the object displays a value equal to the text in the Auxiliary column, the operation is ignored, without reporting the issue.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: YES. You need to tell DASH TEST® what the value you want compare is.
- Uses Component Auxiliary Parameter: YES. You must enter the message that will be ignored by the assertion.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Validate User ID	ASSERT_EQUALS_IF_COMPONENT_PRESENT_AND_NOT_AUX_TEXT	userNm	name	Not Found	JohnSmith

Object File:

userNm=< A 'name' reference in the Web Page using inspector tool >

[Back to DASH TEST® Component Library index](#)

ASSERT_EQUALS_INPUT

DASH TEST® will use this component to perform a validation where, an Input Textbox object in the AUT has the exact same value passed in the CP_01 column. To obtain the value in the Input Textbox, DASH requires the name of the HTML attribute, where that value is stored. Numeric values must be given as TEXT (String). Pay attention to punctuation and letter case, DASH TEST® is case sensitive. DASH will wait for 5 seconds for value to match, before rising an exception, terminate the script and report the issue.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: YES. You need to tell DASH TEST® what the value you want compare is. NOTE: It must be in TEXT format (String).
- Uses Parameter Auxiliary: YES. You must provide the HTML Attribute where the value is stored in the Input Box.

Example:

```
<span class="featured-value-title">Rate</span>
▼<span class="featured-value" id="controlBox">
  <input type="text" name="indexRate" id="indexRate"
  value="280.00" class="text">
  <input type="hidden" name=
```

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Validate Index Rate from Input box	ASSERT_EQUALS_INPUT	indexRateID	id	value	280.00

Object File:

indexRateID=< An 'id' reference in the Web Page using inspector tool >

[Back to DASH TEST® Component Library index](#)

ASSERT_EQUALS_NULL

DASH TEST® will use this component to perform the validation where, the text object in the AUT is blank or NULL. DASH will wait for 5 seconds for value to match, before rising an exception, terminate the script and report the issue.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: NO.

Example:

Script File:

Ex_0	CX_6	SN_1	SD_2	AP_3	OR_4	OT_5	CP_01
Y		1	Validate User ID	ASSERT_EQUALS_NULL	usernameNm	name	

Object File:

usernameNm=< A 'name' reference in the Web Page using inspector tool >

[Back to DASH TEST® Component Library index](#)

ASSERT_EQUALS_TO_STORED_TEXT

DASH TEST® will use this component to perform the assertion where, the text object in the AUT must be the exact same text as a text stored using the component STORE_TEXT. Pay attention to punctuation and letter case, DASH TEST® is case sensitive. DASH will wait for 5 seconds for value to match, before rising an exception, terminate the script and report the issue.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: No.

NOTE: Remember that you must store a text value in DASH TEST® memory before performing this comparison.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Validate User ID	ASSERT_EQUALS	usernameNm	name		

Object File:

usernameNm=< A 'name' reference in the Web Page using inspector tool >

[Back to DASH TEST® Component Library index](#)

ASSERT_EQUALS_TO_STORED_NUMBER

DASH TEST® will use this component to perform the validation where, the text object in the AUT has the exact same numerical value as a number stored using the component STORE_NUMBER. DASH will wait for 5 seconds for value to match, before rising an exception, terminate the script and report the issue.

You can use PERFORM_OPERATION just before, to calculate a value, you want to compare with the AUT, for example a Grand Total.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: No.

NOTE: Remember that you must store a text value in DASH TEST® memory before performing this comparison.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Validate Total Amount	ASSERT_EQUALS_STORED_NUMBER	totalAmountNM	name		

Object File:

totalAmountNM=< A 'name' reference in the Web Page using inspector tool >

[Back to DASH TEST® Component Library index](#)

ASSERT_HTML_ATTRIBUTE_CONTAINS_PARAMETER

DASH TEST® will use this component to perform an assertion where, a Web Page Object's HTML Attribute in the AUT contains the value passed in the CP_01 column. Pay attention to punctuation and letter case, DASH TEST® is case sensitive. DASH will wait for 5 seconds for value to match, before rising an exception, terminate the script and report the issue.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: YES. You need to tell DASH TEST® the text you want compare.
- Uses Parameter Auxiliary: YES. You must provide the HTML Attribute where the tooltip message is.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Validate ToolTip	ASSERT_HTML_ATTRIBUTE_CONTAINS_PARAMETER	objectWithToolTipNm	name	title	Open File

Object File:

objectWithToolTipNm=< A 'name' reference in the Web Page using inspector tool >

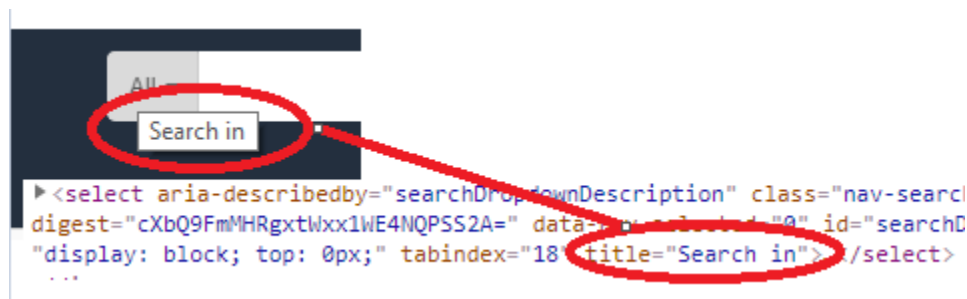
[Back to DASH TEST® Component Library index](#)

ASSERT_HTML_ATTRIBUTE_EQUALS_PARAMETER

DASH TEST® will use this component to perform a validation where, a Web Page Object's Object Attribute in the AUT has the exact same value passed in the CP_01 column. Pay attention to punctuation and letter case, DASH TEST® is case sensitive. DASH will wait for 5 seconds for value to match, before rising an exception, terminate the script and report the issue.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: YES. You need to tell DASH TEST® the text you want compare.
- Uses Parameter Auxiliary: YES. You must provide the HTML Attribute where the tooltip message is.

Example:



Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Validate ToolTip	ASSERT_HTML_ATTRIBUTE_EQUALS_PARAMETER	searchDropID	id	title	Search in

Object File:

searchDropID=< A 'id' reference in the Web Page using inspector tool >

[Back to DASH TEST® Component Library index](#)

ASSERT_NOT_CONTAINS

DASH TEST® will use this component to perform the validation where, the text object in the AUT doesn't contain part of the text passed in the CP_01 column. Use this component to compare values that are not in input text boxes. Pay attention to punctuation and letter case, DASH TEST® is case sensitive. DASH will wait for 5 seconds for value to match, before rising an exception, terminate the script and report the issue.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: YES. You need to tell DASH TEST® what the value you want compare is.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Validate Citizen Kane last words	ASSERT_NOT_CONTAINS	commentsNm	name		Eureka

Object File:

commentsNm=< A 'name' reference in the Web Page using inspector tool >

[Back to DASH TEST® Component Library index](#)

ASSERT_STARTS_WITH

DASH TEST® will use this component to perform the assertion where, the text object in the AUT exactly starts with the text passed in the CP_01 column. Use this component to compare values that are not in input text boxes. Pay attention to punctuation and letter case, DASH TEST® is case sensitive. DASH will wait for 5 seconds for value to match, before rising an exception, terminate the script and report the issue.

- Uses Object Reference: YES, a reference to an object in the Object Repository.
- Uses Object Type: YES. Must tell if must be found by ID, XPath or Name.
- Uses Component Parameter: YES. You need to tell DASH TEST® what the value you want compare is.

Example:

Script File:

Ex_0	SN_1	SD_2	AP_3	OR_4	OT_5	CX_6	CP_01
Y	1	Assert Declaration of Independence	ASSERT_STARTS_WITH	declIndepNm	name		We the people

Object File:

declIndepNm=< A 'name' reference in the Web Page using inspector tool >

[Back to DASH TEST® Component Library index](#)

The DASH TEST© XML Suite Configuration File

This is an example of a typical XML Test Suite configuration file for DASH TEST©, which will run only one test case:

```
<?xml version="1.0" encoding="UTF-8"?>
<!--DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd"-->
<suite name="testSuite">
  <!-- This is a comment -->
  <test verbose= "2" name="Test Number 1">
    <parameter name="webBrowser" value="CHROME"/>
    <parameter name="scenarioFilePath" value="ScriptFile.xls"/>
    <parameter name="testCaseID" value="WorkSheetName"/>
    <parameter name="testCaseName" value="Test Case Title"/>
    <parameter name="objectFilePath" value="objectLibrary.properties"/>
  <classes>
    <class name="dASH_Main.DASH"/>
  </classes>
</test>
</suite>
```

Below is the description of the Test Suite configuration File components:

suite name="Test Suite Name": This statement contains the name of the Test Suite that will be executed (testSuite). In addition, this is the name of the folder where test results and logs will be placed under the test-output directory. You can give it any name you want. The complete Test Suite must be delimited by the <suite></suite> delimiters.

test verbose= "2" name="Test Name": This statement contains 2 parameters. “verbose” is used to tell the level of detail passed to the log file, the recommendation is to leave it in level 2. “name” contains the short version of the Test Case name. Since the user can execute multiple test scripts of the same application/user session with DASH TEST©, this is a unique name given to identify each test configured to run. Each test group must be enclosed by the <test> </test> delimiters.

There are 5 parameters that must be passed to DASH, to have it execute your test scripts. Each parameter is referenced with the “parameter name” clause and the value is passed with the “value” clause.

Parameters passed to DASH TEST©:

parameter name="webBrowser" value="BrowserName": This is the Web Browser that will be used to run the tests. Only one web browser can be specified per Test Suite. If there are more than one tests in the test suite, the first test will tell what web browser will be used and the rest will have the value of “SAME”, telling DASH to maintain execution using the same Selenium WebDriver object.

Allowed values for these parameters are: CHROME, FIREFOX, IE and SAME.

Example:

```
<parameter name="webBrowser" value="CHROME"/>
```

parameter name="scenarioFilePath" value="ScriptFileName.xls": The value passed to by this parameter is the location and the name of the test scenario file containing the steps DASH TEST® will perform. It is case sensitive. If the Test Scenario file is in a directory structure inside the automatedScripts directory.

Examples:

```
<parameter name="scenarioFilePath" value="Folder1\SubFolder1.1\...\ScriptFile.xls"/>
```

```
<parameter name="scenarioFilePath" value="Folder1\SubFolder1.1\...\ScriptFile.xlsx"/>
```

Note that it is very important to include the Excel File extension, because DASH TEST® needs this information to identify if it is an MS Excel '97-2003 workbook (xls) or a later version (xlsx). Internal operations that process script files are different, from one type to the other. Macro (xlsm), template (xlt), Comma Delimited files (csv) or **any other format different than XLS or XLSX, are not supported by DASH TEST®.**

Though, XLSX files are supported, DASH TEST® performs better with the XLS (MS Excel '97-2003) format.

parameter name="testCaseID" value="WorksheetName": This parameter contains the name of the individual worksheet where the test script is contained inside the test scenario file. It is case sensitive, cannot contain spaces or special characters. Size is limited to 31 characters.



parameter name="testCaseName" value="Test Case Title": This is the long version of the Test Case name. There are no restrictions for this value.

Example:

```
<parameter name="testCaseName" value="This can be any Test Case Title you want to display"/>
```

parameter name="objectFilePath" value="objectLibrary.properties": This parameter contains the name of the object repository and object library file. It is case sensitive, cannot contain spaces or special characters. File name and reference must end with the extension ".properties".

Example:

```
<parameter name="objectFilePath" value="objectLibrary.properties"/>
```

Comments: They must be enclosed inside the <!-- and --> delimiters.

class name="dASH_Main.DASH": This is the call to the main class of DASH. The call to DASH main class must be enclosed inside the <classes></classes> delimiters.

NOTE: To successfully use FIREFOX as your testing web browser, create a *webdriver* profile in the Profile Manager:

1. Click the Start button
2. Click "Run"
3. Type "firefox -ProfileManager"

For more information and details for your current version of Firefox, check the Firefox Help files.

Execute DASH TEST© from command prompt

To execute DASH TEST© from the command prompt, use the following syntax depending on your operating system.

NOTE: For convenience, place your configuration files and script files in the automatedScripts path.

Windows:

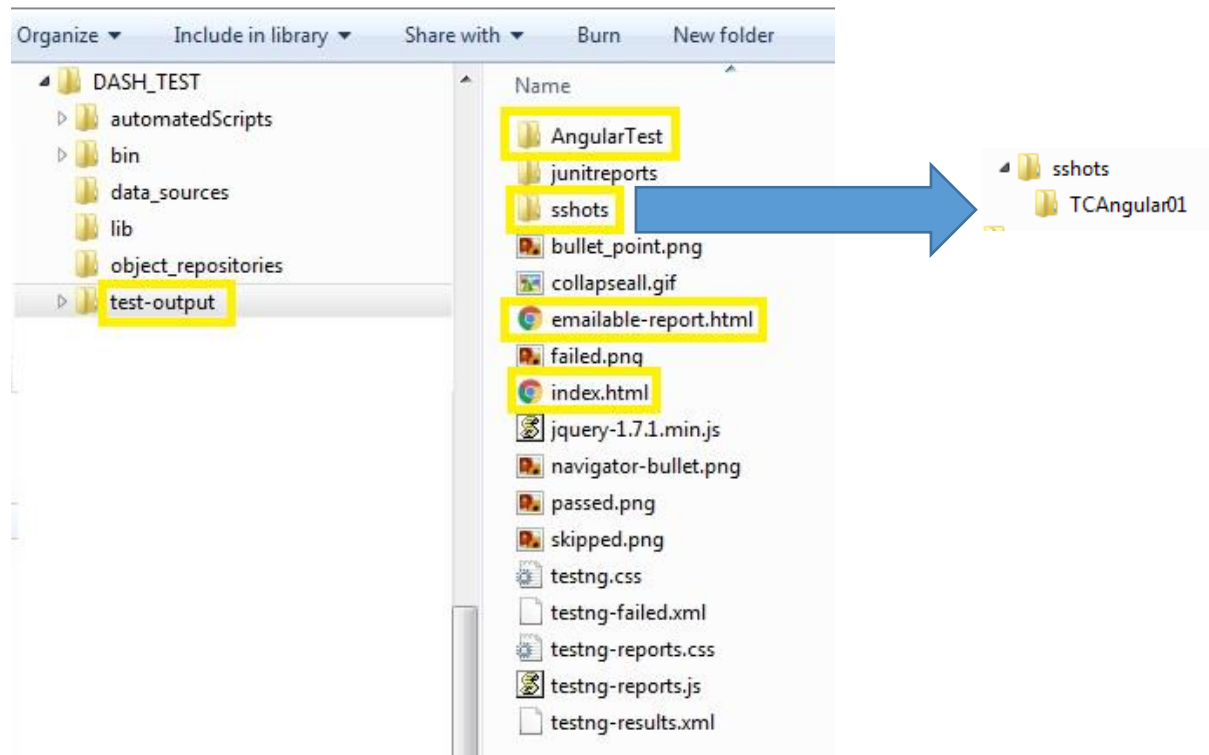
```
java -cp "lib\*;bin" org.testng.TestNG automatedScripts\[pathToConfigFile\]configFileName.xml>
```

Linux/MacOS:

```
java -cp "./lib/*;bin" org.testng.TestNG ./automatedScripts/[pathToConfigFile/]configFileName.xml>
```

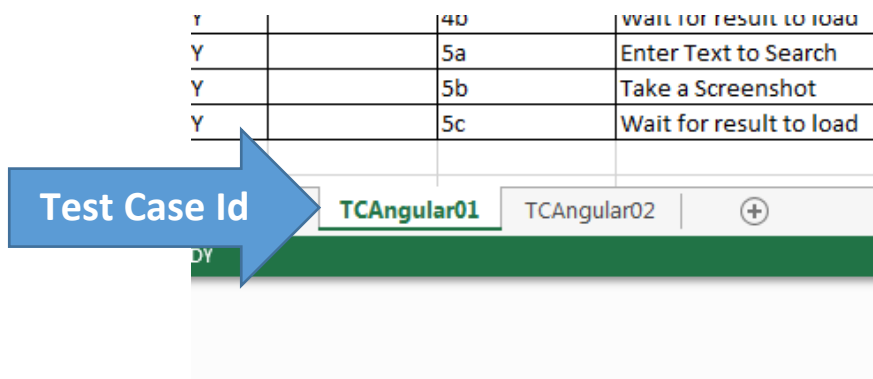

Understanding “DASH TEST©” test Reports

The following image shows the typical contents of the test-output directory, where DASH TEST® stores test results.



Test Suite Directory (example: “AngularTest” folder): This directory contains execution reports of each of the test components in the test suite. The name of this directory is extracted from the Test Configuration file, at the “suite name” line.

Screenshots (“sshots” folder): This directory contains all the screenshots taken per Test Case script. The name of the subdirectory, where screenshots are stored is taken from the Test Case Id or Worksheet Name.



The E-mailable Report

This file (emailable-report.html) contains a text formatted form of report, of the Test Suite Execution. The report provides information about the execution status of each of the components in the Test Suite in a single file without screenshots.

The report contains the following components:

Execution Results Table: A table showing the Passed/Failed status of each test in the test suite. In addition, displays the time taken to complete each test and the test suite.

Test	# Passed	# Skipped	# Failed	Time (ms)	Included Groups	Excluded Groups
AngularTest						
Test_01	1	0	0	26,290		
Test_02	1	0	0	3,544		
Total	2	0	0	29,834		

Test Case Result Status: Each test execution status, using color code: Green, pass; Red, fail.

Class	Method	Start	Time (ms)
AngularTest			
Test_01 — passed			
DASH_Main.DASH	testExecutionControl	1452896349562	26271
Test_02 — passed			
DASH_Main.DASH	testExecutionControl	1452896375890	3543

Execution Details: A log showing each step taken during Test Case execution, Test Case by Test Case. The table header shows the Web Browser used, the path to the Test Case file, the name of the Test Case Id, a brief description of the Test Case, the object properties file used to locate the objects in the AUT.

Test_01

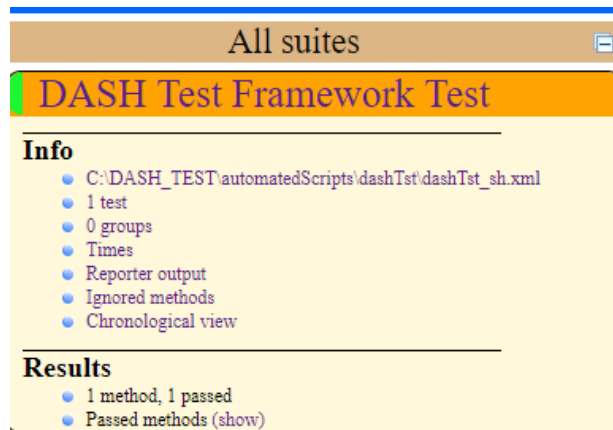
DASH_Main.DASH#testExecutionControl

Parameter #1	Parameter #2	Parameter #3	Parameter #4	Parameter #5
CHROME	angular\TS-Angular01.xls	TCAngular01	This test case will test an Angular made Web Page	angular_objects.properties
Messages				
DASH is a CopyRight property of: Edgar Chavez 2015 (PRE000008332)... Designed and Developed by Edgar Chavez. DASH TEST... DASHING! Loading CHROME ***** Executing Test Case (TCAngular01): "This test case will test an Angular made Web Page" =====				
Starting Iteration #1 =====				
Step Number 1a: Navigate to URL -----				
Navigate to -> urlAngular01 =====				

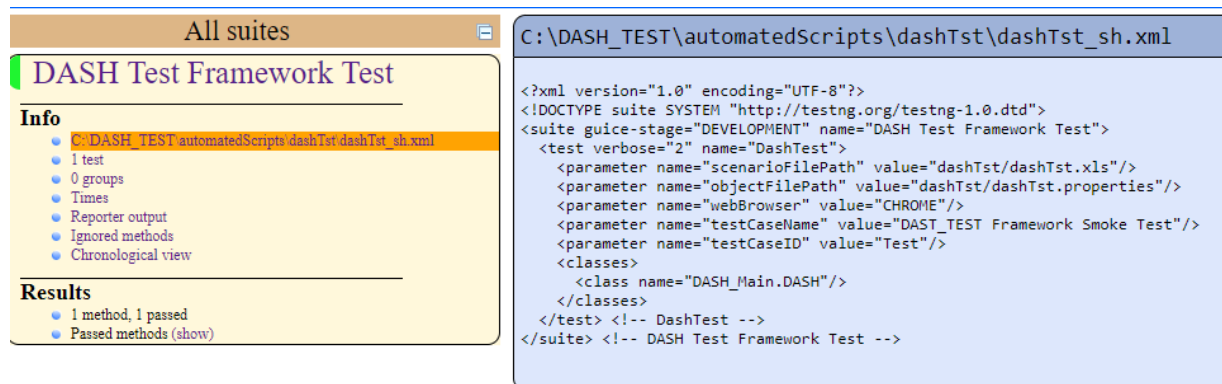
The Full Report

The full report (index.html) is an HTML graphical version of the emailable report. This version contains expandable sections and can contain screenshots.

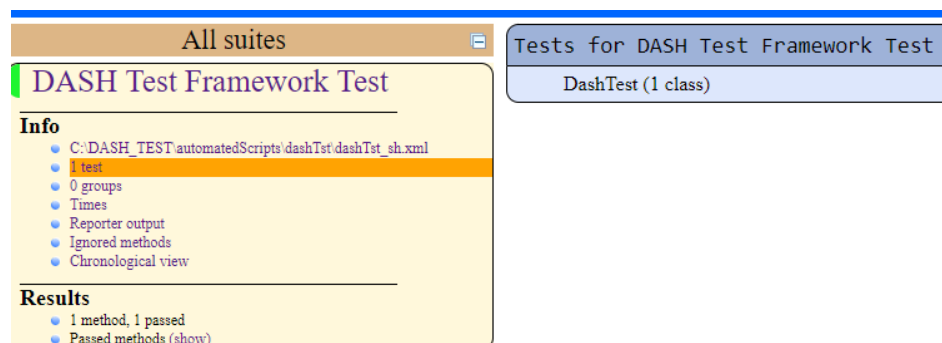
On the left side of the report, the control frame displays the Information (Info) section and the Results section.



1. In the Info section the first link displays full the path and name of the DASH TEST[®] configuration file, containing the complete executed test suite.
Clicking on that link, displays the contents of the configuration file.



2. The second link, when clicked, displays the test that were executed in the test suite.



3. The third link (not used), displays the groups of tests.
4. The fourth link displays the time taken by each test to execute.

All suites

DASH Test Framework Test

Info

- C:\DASH_TEST\automatedScripts\dashTst\dashTst_sh.xml
- 1 test
- 0 groups
- Times**
- Reporter output
- Ignored methods
- Chronological view

Results

- 1 method, 1 passed
- Passed methods (show)

Times for DASH Test Framework Test

Total running time: 14 seconds			
Number	Method	Class	Time (ms)
0	testExecutionControl	DASH_Main.DASH	14,173

5. The fifth link is the complete Reporter output, listing all steps in the execution.

All suites

DASH Test Framework Test

Info

- C:\DASH_TEST\automatedScripts\dashTst\dashTst_sh.xml
- 1 test
- 0 groups
- Times
- Reporter output**
- Ignored methods
- Chronological view

Results

- 1 method, 1 passed
- Passed methods (show)

Reporter output for DASH Test Framework Test

```
testExecutionControl(CHROME, dashTst/dashTst.xls, Test, DASH_TEST
Framework Smoke Test, dashTst/dashTst.properties)

*****

DASH is a CopyRight property of: Edgar Chavez 2015 (PRE000008332)...
Designed and Developed by Edgar Chavez.
DASH TEST... DASHING!
Test Executed on: 2019/02/27 11:10:26
Loading CHROME
*****

Executing Test Case (Test): "DAST_TEST Framework Smoke Test"
*****

Starting Iteration #1
*****

Step Number 1a: Navigate to URL
----- Navigate to -> urlTest01URL
*****

Step Number 1b: Verify Page Loaded
*****

Step Number 1c: Store Text from Random Search button
-----
Using CSS Property: aria-label
Stored Value/Property value: I'm Feeling Lucky
*****

Step Number 2.0: Enter Text to Search
----- Data Entered: : Man on the moon
*****

Step Number 3a: Press the Google Search button
*****

Step Number 3b: Verify Page Loaded
*****
```

6. The sixth link (not used), displays ignored tests.
7. The seventh link displays the chronological list in time for each test in the test suite.

All suites

DASH Test Framework Test

Info

- C:\DASH_TEST\automatedScripts\dashTst\dashTst_sh.xml
- 1 test
- 0 groups
- Times
- Reporter output
- Ignored methods
- Chronological view**

Results

- 1 method, 1 passed
- Passed methods (show)

Methods in chronological order

DASH_Main.DASH

```
testExecutionControl(CHROME,
dashTst/dashTst.xls, Test, DASH_TEST
Framework Smoke Test,
dashTst/dashTst.properties)
```

0 ms

At the results section, the status of the test displays.

In this case the test passed successfully.

The screenshot shows the 'All suites' tab with the 'DASH Test Framework Test' suite selected. The 'Info' section lists the test path and configuration. The 'Results' section shows '1 method, 1 passed' and 'Passed methods (hide)'. The test execution details on the right show a successful run of 'testExecutionControl'.

All suites

DASH Test Framework Test

Info

- C:\DASH_TEST\automatedScripts\dashTst\dashTst_sh.xml
- 1 test
- 0 groups
- Times
- Reporter output
- Ignored methods
- Chronological view

Results

- 1 method, 1 passed
- Passed methods (hide)
- **testExecutionControl(CHROME, dashTst\dashTst.xls, Test, DAST_TEST Framework Smoke Test, dashTst\dashTst.properties)**

DASH_Main.DASH

```
testExecutionControl (CHROME, dashTst\dashTst.xls,
Test, DAST_TEST Framework Smoke Test,
dashTst\dashTst.properties)
```

In this case the test failed. On the right, a detailed description of the error.

The screenshot shows the 'All suites' tab with the 'DASH Test Framework Test' suite selected. The 'Info' section lists the test path and configuration. The 'Results' section shows '1 method, 1 failed' and 'Failed methods (hide)'. The test execution details on the right show a failed run of 'testExecutionControl' with a detailed stack trace.

All suites

DASH Test Framework Test

Info

- C:\DASH_TEST\automatedScripts\dashTst\dashTst.xml
- 1 test
- 0 groups
- Times
- Reporter output
- Ignored methods
- Chronological view

Results

- 1 method, 1 failed
- Failed methods (hide)
- **testExecutionControl(CHROME, dashTst\dashTst.xls, Test, DAST_TEST Framework Smoke Test, dashTst\dashTst.properties)**

DASH_Main.DASH

```
testExecutionControl (CHROME, dashTst\dashTst.xls, Test, DAST_TEST
Framework Smoke Test, dashTst\dashTst.properties)
java.lang.AssertionError:
  An Exception Occurred!
    at org.testng.Assert.fail(Assert.java:83)
    at operation.UIOperation.store_Text(UIOperation.java:934)
    at operation.UIOperation.dashActions(UIOperation.java:152)
    at DASH_Main.DASH.testExecutionControl(DASH.java:137)
    at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
    at sun.reflect.NativeMethodAccessorImpl.invoke(Unknown Source)
    at sun.reflect.DelegatingMethodAccessorImpl.invoke(Unknown Source)
    at java.lang.reflect.Method.invoke(Unknown Source)
    at org.testng.internal.MethodInvocationHelper.invokeMethod(MethodInvocationHelper.java:85)
    at org.testng.internal.Invoker.invokeMethod(Invoker.java:639)
    at org.testng.internal.Invoker.invokeTestMethod(Invoker.java:821)
    at org.testng.internal.Invoker.invokeTestMethods(Invoker.java:1131)
    at org.testng.internal.TestMethodWorker.invokeTestMethods(TestMethodWorker.java:124)
    at org.testng.internal.TestMethodWorker.run(TestMethodWorker.java:108)
    at org.testng.TestRunner.privateRun(TestRunner.java:773)
    at org.testng.TestRunner.run(TestRunner.java:623)
    at org.testng.SuiteRunner.runTest(SuiteRunner.java:357)
    at org.testng.SuiteRunner.runSequentially(SuiteRunner.java:352)
    at org.testng.SuiteRunner.privateRun(SuiteRunner.java:310)
    at org.testng.SuiteRunner.run(SuiteRunner.java:259)
    at org.testng.SuiteRunnerWorker.runSuite(SuiteRunnerWorker.java:52)
    at org.testng.SuiteRunnerWorker.run(SuiteRunnerWorker.java:86)
    at org.testng.TestNG.runSuitesSequentially(TestNG.java:1185)
    at org.testng.TestNG.runSuitesLocally(TestNG.java:1110)
    at org.testng.TestNG.run(TestNG.java:1018)
    at org.testng.TestNG.privateMain(TestNG.java:1325)
    at org.testng.TestNG.main(TestNG.java:1294)
Caused by: java.lang.Exception:
  Wrong object type
    at operation.UIOperation.getObject(UIOperation.java:1491)
    at operation.UIOperation.store_Text(UIOperation.java:925)
    ... 25 more
```

The error “Wrong object type”, refers to when in the excel file, the Component Type (OT_5) was not specified for one of the actions that required it or when a Text parameter was expected, but a Number type was passed.